

# **COMPLAINT CARE - A COMPLAINT MANAGEMENT SYSTEM WITH AI CHATBOT INTEGRATION & TRANSFORMER-BASED ARCHITECTURE FOR TICKET MANAGEMENT SYSTEM**

**CAPSTONE PROJECT REPORT**

**Submitted by**

**G. Bharadwaj Naidu – 99220040844**

**B. Rami Reddy – 99220040816**

**G. Harshavardhan Reddy – 99220040858**

**K. Mahidhar Reddy– 99220040868**

**in partial fulfillment for the award of the degree  
of**

**BACHELOR OF TECHNOLOGY**

**IN**

**COMPUTER SCIENCE AND ENGINEERING**



**SCHOOL OF COMPUTING**

**COMPUTER SCIENCE AND ENGINEERING**

**KALASALINGAM ACADEMY OF RESEARCH AND EDUCATION**

**KRISHNANKOIL 626 126**

**April 2026**

## DECLARATION

We affirm that the project work titled “**COMPLAINT CARE - A COMPLAINT MANAGEMENT SYSTEM WITH AI CHATBOT INTEGRATION & TRANSFORMER-BASED ARCHITECTURE FOR TICKET MANAGEMENT SYSTEM**” being submitted in partial fulfillment for the award of the degree of **Bachelor of Technology in Computer Science and Engineering** is the original work carried out by us. It has not formed part of any other project work submitted for the award of any degree or diploma, either in this or any other University.

G.Bharadwaj Naidu

99220040844

B.Rami Reddy

99220040816

G.Harshavardhan Reddy

99220040858

K.Mahidhar Reddy

99220040868

This is to certify that the above statement made by the candidate is correct to the best of my knowledge.

Date:

Signature of supervisor

**Dr. T. Manikumar**

**Associate Professor**

**Department of Computer Science and Engineering**



**KALASALINGAM**  
**ACADEMY OF RESEARCH AND EDUCATION**  
**(DEEMED TO BE UNIVERSITY)**  
Under sec. 3 of UGC Act 1956. Accredited by NAAC with "A++" Grade



## **BONAFIDE CERTIFICATE**

Certified that this project report titled “COMPLAINT CARE - A COMPLAINT MANAGEMENT SYSTEM WITH AI CHATBOT INTEGRATION & TRANSFORMER-BASED ARCHITECTURE FOR TICKET MANAGEMENT SYSTEM” is the bonafide work of **G. Bharadwaj Naidu (99220040844), B. Ramireddy (99220040816), G. Harsha Vardhan Reddy (99220040858), and K. Mahidhar Reddy (99220040868)**, who carried out the project work under my supervision.

**Dr. T. Manikumar**

**SUPERVISOR**

**Associate Professor**

Computer Science and Engineering  
Kalasalingam Academy of Research and  
Education  
Krishnankoil 626126  
Virudhunagar District.

**Dr. R. Raja Subramanian**

**HEAD OF THE DEPARTMENT**

**Associate Professor & Head**

Computer Science and Engineering  
Kalasalingam Academy of Research and  
Education  
Krishnankoil 626126  
Virudhunagar District.

Submitted for the Project Viva-voce examination held on .....

**Internal Examiner**

**External Examiner**

## ACKNOWLEDGEMENT

We would like to begin by expressing our heartfelt gratitude to the Supreme Power for the immense grace that enabled us to complete this project.

We are deeply grateful to the late "**Kalvivallal**" **Thiru T. Kalasalingam**, Chairman of the Kalasalingam Group of Institutions, and to the late "**Illayavallal**" **Dr. K. Sridharan**, Chancellor, as well as **Dr. S. Shasi Anand**, Vice President, who has been a guiding light in all our university's endeavours.

Our sincere thanks go to our Vice Chancellor, **Dr. S. Narayanan**, for his inspiring leadership, guidance, and for instilling in us the strength and enthusiasm to work towards our goals.

We would like to express our sincere appreciation to **Dr. P. Deepalakshmi**, Professor & Dean-(SoC), Director Accreditation & Ranking, for her valuable guidance. Our heartfelt gratitude also goes to our esteemed Head of Department, **Dr. R. Raja Subramanian**, whose unwavering support has been crucial to the successful advancement of our project.

We are especially thankful to our Project Supervisor, **Dr. T. Manikumar**, for his patience, motivation, enthusiasm, and vast knowledge, which greatly supported us throughout this work.

Our sincere gratitude also goes to **Dr. T. Manikumar** and **Mr. N. R. Sathiskumar** Overall Project Coordinators, for their constant encouragement and support in completing this Capstone Project.

Finally, we would like to thank our parents, faculty, non-teaching staff, and friends for their unwavering moral support throughout this journey.



**SCHOOL OF COMPUTING**  
**COMPUTER SCIENCE AND ENGINEERING**  
**PROJECT SUMMARY**

|                                                                |                                                                                                                                                                                                                                                             |        |
|----------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|
| Project Title                                                  | Complaint Care - A Complaint Management System With AI Chatbot Integration & Transformer-based Architecture For Ticket Management System                                                                                                                    |        |
| Project Team Members (Name with Register No)                   | G. Bharadwaj Naidu – 99220040844<br>B. Rami Reddy– 99220040816<br>G. Harsha Vardhan Reddy – 99220040858<br>K. Mahidhar Reddy – 99220040868                                                                                                                  |        |
| Guide Name/Designation                                         | <b>Dr. T. Manikumar</b><br>Associate Professor, Computer Science and Engineering,<br>Kalasalingam Academy of Research and Education.                                                                                                                        |        |
| Program Concentration Area                                     | Full-stack development using the ReactJS Python Flask, Machine learning.                                                                                                                                                                                    |        |
| Technical Requirements                                         | MERN stack (MongoDB, Express.js, React.js, Node.js), Deep Translator, Gemini API, Swin Transformer.                                                                                                                                                         |        |
| Engineering standards and realistic constraints in these areas |                                                                                                                                                                                                                                                             |        |
| Area                                                           | Codes & Standards / Realistic Constraints                                                                                                                                                                                                                   | Tick ✓ |
| Economic                                                       | The project involves costs related to cloud services (Neon database, Cloudinary), API usage (Gemini chatbot), and deployment platforms (Vercel, Docker). Budget limitations may restrict scalability, premium API usage, and advanced infrastructure setup. | ✓      |
| Environmental                                                  | Although the system is digital and reduces paper-based complaint handling, it indirectly contributes to energy consumption through servers, cloud storage, and AI model processing, which must be optimized for efficiency.                                 | ✓      |
| IEEE Standards                                                 | The project follows IEEE 830, IEEE 1016, IEEE 730, and IEEE 29148 standards to ensure proper requirement specification, system design, software quality, and requirement traceability.                                                                      | ✓      |

## **REALISTIC CONSTRAINTS:**

### **Economic:**

The project involves significant costs related to cloud-based infrastructure such as Neon for database management, Cloudinary for media storage, and external APIs like Gemini for chatbot integration, along with deployment platforms such as Vercel and Docker. Proper cost optimization can limit the use of premium services, large-scale deployment, and high-performance infrastructure. Proper cost optimization strategies are required to balance performance, scalability, and budget limitations.

### **Environmental:**

Although the system eliminates the need for paper-based complaint handling and promotes digital efficiency, it indirectly contributes to environmental impact through energy consumption in data centers, cloud storage, and AI model computations. Therefore, optimizing resource utilization, reducing unnecessary computations, and adopting efficient coding practices are essential to minimize unnecessary computations, and adopting efficient coding practices are essential to minimize environmental impact.

### **Engineering Standards:**

The project strictly adheres to established IEEE standards, including IEEE 830 (Software Requirements Specification), IEEE 1016 (Software Design Description), IEEE 730 (Software Quality Assurance), and IEEE 29148 (Requirements Engineering), to ensure clear requirement definition, structured system design, and high software quality. This adherence enables a systematic and well-organized development process with proper documentation, reliability, maintainability, and traceability across all stages of the project lifecycle. Furthermore, it minimizes development risks, enhances scalability, and aligns the system with globally accepted industry best practices, making it suitable for real-world deployment and future enhancements.

## ABSTRACT

Among the major challenges faced by organizations is the efficient handling of customer complaints, which directly impacts service quality, operational efficiency, and user satisfaction. Traditional complaint management systems rely on manual processes for logging, categorizing, and resolving tickets, which are often time-consuming, error-prone, and inconsistent, especially in high-volume environments. In this regard, this study aims to develop an automated complaint management system that integrates an AI chatbot with a transformer-based architecture for intelligent ticket classification and management.

The proposed framework leverages the capabilities of transformer models, utilizing self-attention mechanisms to understand the contextual meaning of user complaints and extract both local and global dependencies in textual data. The system is trained and evaluated on complaint datasets after applying preprocessing techniques to enhance model performance and generalization. Experimental results demonstrate high accuracy, precision, recall, and F1-score in complaint classification and prioritization. Furthermore, the analysis of the confusion matrix validates the reliability and effectiveness of the system in handling diverse complaint categories.

In addition to classification, the system enhances accessibility and user interaction by integrating an AI chatbot that supports real-time communication and multilingual responses. A user-friendly interface is developed to allow complaint submission, tracking, and visualization of ticket status and analytics. The system is designed as a decision support tool to assist organizations rather than replace human intervention. Future enhancements include incorporating explainable AI techniques, voice-based complaint handling, and advanced predictive analytics for proactive issue resolution.

**Keywords** – Complaint Management System, AI Chatbot, Transformer Architecture, Ticket Classification, Natural Language Processing, Multilingual Support, Automated Customer Service, Real-Time Monitoring.

## TABLE OF CONTENTS

| TITLE                                     |                                                                                                                 | PAGE NO. |
|-------------------------------------------|-----------------------------------------------------------------------------------------------------------------|----------|
| <b>ABSTRACT</b>                           |                                                                                                                 | vii      |
| <b>LIST OF TABLES</b>                     |                                                                                                                 | xi       |
| <b>LIST OF FIGURES</b>                    |                                                                                                                 | xii      |
| <b>LIST OF ACADEMIC REFERENCE COURSES</b> |                                                                                                                 | xiii     |
| <b>CHAPTER I</b>                          | <b>INTRODUCTION</b>                                                                                             | 1        |
| 1.1                                       | Background and Motivation                                                                                       |          |
| 1.2                                       | Problem Statement                                                                                               |          |
| 1.3                                       | Objectives of the Project                                                                                       |          |
| 1.4                                       | Scope of the Project                                                                                            |          |
| 1.5                                       | Methodology Overview                                                                                            |          |
| 1.6                                       | Organization of the Report                                                                                      |          |
| <b>CHAPTER II</b>                         | <b>LITERATURE REVIEW</b>                                                                                        | 8        |
| 2.1                                       | Overview of Related Work                                                                                        | 8        |
| 2.2                                       | Review of Similar Projects or Research Papers                                                                   | 9        |
| 2.3                                       | Summary and Gap Identification                                                                                  | 10       |
| <b>CHAPTER III</b>                        | <b>SYSTEM ANALYSIS</b>                                                                                          | 11       |
| 3.1                                       | Requirements Gathering                                                                                          | 11       |
| 3.2                                       | Functional Requirements                                                                                         | 12       |
| 3.3                                       | Non-Functional Requirements                                                                                     | 13       |
| 3.4                                       | Feasibility Study<br>3.4.1 Technical Feasibility<br>3.4.2 Operational Feasibility<br>3.4.3 Economic Feasibility | 14       |
| 3.5                                       | Risk Analysis                                                                                                   | 18       |
| <b>CHAPTER IV</b>                         | <b>SYSTEM DESIGN</b>                                                                                            | 19       |
| 4.1                                       | Overall System Architecture                                                                                     | 20       |
| 4.2                                       | Module Design                                                                                                   | 20       |
| 4.2.1                                     | Module 1 - Image Upload and Preprocessing                                                                       | 20       |
| 4.2.2                                     | Module 2 - Classification and Summary Generation                                                                | 21       |
| 4.2.3                                     | Module 3 - Multilingual Summary Generation Module                                                               | 21       |

|                          |                                                           |    |
|--------------------------|-----------------------------------------------------------|----|
| 4.2.4                    | Module 4 - Result Visualization and User Interface Module | 21 |
| 4.3                      | Database Design                                           | 22 |
| 4.3.1                    | ER Diagram                                                | 22 |
| 4.3.2                    | Database Schema                                           | 23 |
| 4.4                      | User Interface Design                                     | 23 |
| 4.4.1                    | User Flow Diagrams                                        | 24 |
| <b>CHAPTER V</b>         | <b>IMPLEMENTATION</b>                                     | 28 |
| 5.1                      | Technology Stack                                          | 28 |
| 5.1.1                    | Programming Languages and Tools                           | 29 |
| 5.2                      | Implementation of Modules                                 | 30 |
| 5.2.1                    | Module 1 - User Interaction and Preprocessing Module      | 30 |
| 5.2.2                    | Module 2 - Classification and Multilingual Summary Module | 31 |
| 5.3                      | Integration of Modules                                    | 32 |
| <b>CHAPTER VI</b>        | <b>TESTING</b>                                            | 33 |
| 6.1                      | Testing Methodology                                       | 34 |
| 6.1.1                    | Unit Testing                                              | 35 |
| 6.1.2                    | Integration Testing                                       | 36 |
| 6.1.3                    | System Testing                                            | 37 |
| 6.1.4                    | User Acceptance Testing (UAT)                             | 38 |
| 6.2                      | Test Cases and Results                                    | 38 |
| 6.3                      | Bug Tracking and Resolution                               | 39 |
| <b>CHAPTER VII</b>       | <b>RESULTS AND DISCUSSION</b>                             | 40 |
| 7.1                      | System Output Screenshots                                 | 40 |
| 7.2                      | Evaluation Metrics                                        | 42 |
| 7.3                      | Comparison with Existing Systems                          | 43 |
| 7.4                      | Challenges Faced                                          | 44 |
| 7.5                      | Solutions and Improvements                                | 45 |
| <b>CHAPTER VIII</b>      | <b>CONCLUSION &amp; FUTURE SCOPE</b>                      | 47 |
| <b>REFERENCES</b>        |                                                           | 48 |
| <b>PUBLICATION</b>       |                                                           |    |
| <b>CERTIFICATIONS</b>    |                                                           |    |
| <b>PLAGIARISM REPORT</b> |                                                           |    |

## LIST OF TABLES

| <b>TABLES</b> | <b>DETAILS</b>                                            | <b>PAGE NO.</b> |
|---------------|-----------------------------------------------------------|-----------------|
| Table 1       | Precision, Recall, F1 Score for Different components used | 41              |
| Table 2       | Accuracy and Quality Metrics                              | 41              |

## LIST OF FIGURES

| <b>FIGURES</b> | <b>DETAILS</b>        | <b>PAGE NO.</b> |
|----------------|-----------------------|-----------------|
| Figure 1       | User flow Diagram     | 24              |
| Figure 2       | Use Case Diagram      | 39              |
| Figure 3       | Class Diagram         | 38              |
| Figure 4       | Sequence Diagram      | 39              |
| Figure 5       | Landing Page          | 40              |
| Figure 6       | User Login            | 41              |
| Figure 7       | User Dashboard        | 41              |
| Figure 8       | Complaint submissions | 42              |
| Figure 9       | Model Comparisons     | 42              |

## **LIST OF ACADEMIC REFERENCE COURSES**

| <b>S. NO.</b> | <b>COURSE CODE</b> | <b>COURSE NAME</b>          |
|---------------|--------------------|-----------------------------|
| 1             | 212CSE2304         | Machine Learning            |
| 2             | 213CSE3301         | Deep Learning               |
| 3             | 213CSE3303         | Natural Language Processing |
| 4             | 211CSE1402         | Python Programming          |

# CHAPTER - I

## INTRODUCTION

### 1.1 Background and Motivation

Customer complaints represent critical feedback that reflects user dissatisfaction, service inefficiencies, and operational gaps within an organization. Efficient handling and timely resolution of these complaints are essential to maintain customer trust, improve service quality, and ensure organizational growth. However, traditional complaint management systems rely heavily on manual processes, where support staff are responsible for logging, categorizing, and resolving tickets. This approach is often time-consuming, prone to human error, and highly dependent on individual expertise, leading to inconsistencies, especially in high-volume environments.

The conventional method of handling complaints becomes increasingly inefficient in organizations dealing with large-scale user interactions or limited support resources. Delays in identifying and resolving issues can negatively impact customer satisfaction and brand reputation. In such scenarios, there is a strong need for intelligent systems that can automate and optimize the complaint management process.

Recent advancements in Artificial Intelligence (AI) and Natural Language Processing (NLP) have shown promising results in automating text-based tasks such as complaint classification and response generation. Traditional machine learning models have been used for these tasks; however, they often struggle to capture complex contextual relationships and long-range dependencies in textual data. Transformer-based architectures address these limitations by leveraging self-attention mechanisms to understand both local and global context, enabling more accurate and efficient processing of user complaints.

Moreover, many existing automated systems provide only basic responses or ticket status updates, which may not be easily understandable to all users. In multilingual environments, the lack of language support further limits accessibility and usability. This highlights the need to design a system that not only automates complaint classification and ticket management but also enhances user interaction through an AI chatbot capable of providing clear, context-aware, and multilingual responses.

## 1.2 Problem Statement

The management of customer complaints in organizations is a complex and time-consuming process that heavily depends on manual intervention by support staff. Human handling of complaints can lead to inconsistencies due to errors, workload pressure, or subjective judgment. In organizations with a high volume of user interactions, delays in complaint resolution can negatively impact customer satisfaction, service quality, and overall brand reputation. Timely identification and resolution of issues are critical, yet the availability of skilled personnel and efficient systems remains a challenge in many environments. These limitations highlight the need for automated solutions that can assist in faster, more accurate, and consistent complaint handling, reducing reliance on manual processes.

Although traditional machine learning and rule-based systems have been used for complaint classification and ticket management, they suffer from several limitations. These models often rely on predefined rules or simple feature extraction techniques, making them less effective in understanding the complex context and intent behind user complaints. Complaints can vary significantly in language, tone, and structure, requiring models capable of capturing both local and global contextual relationships. Conventional approaches may fail to accurately classify such diverse inputs, leading to improper ticket routing and delayed resolution. Additionally, many existing systems lack scalability and adaptability across different domains and datasets.

Another significant challenge in current complaint management systems is the lack of intelligent interaction and user-friendly interfaces. Most systems require users to manually submit structured forms, which can be inconvenient and time-consuming. Furthermore, these systems often do not provide clear explanations or real-time assistance, making it difficult for users to understand the status or resolution of their complaints. In multilingual environments, the absence of language support further reduces accessibility, limiting the usability of such systems for a diverse user base.

Moreover, many existing solutions focus primarily on ticket logging and tracking without addressing the overall user experience and practical deployment challenges. There exists a gap between technically efficient systems and real-world usability, where factors such as response time, intuitive interfaces, automated assistance, and multilingual communication play a crucial role. Integrating an AI-powered chatbot with advanced transformer-based models can bridge this gap by enabling intelligent complaint handling, real-time interaction, and automated ticket management. Therefore, there is a strong need for a comprehensive system that combines

accuracy, scalability, interpretability, and user-centric design to improve complaint management processes effectively.

### **1.3 Objectives of the Project**

The main goal of this project is to develop an automated complaint management system that leverages a transformer-based architecture for efficient ticket classification and management. The system will be capable of analyzing user complaints and categorizing them into appropriate classes such as technical issues, service requests, billing problems, or general inquiries. By utilizing transformer models, the system aims to effectively understand both the contextual meaning and semantic relationships in user inputs, ensuring high accuracy in classification, prioritization, and routing of tickets. The primary objective is to achieve high precision, recall, and F1-score values in complaint classification tasks.

Another significant objective of this project is to implement effective text preprocessing and data handling techniques. Preprocessing steps such as text cleaning, tokenization, and normalization are essential for improving model performance and generalization. The system will be designed to handle variations in user inputs, including differences in language, grammar, tone, and structure. The objective is to ensure consistent and high-quality input representation for the transformer model.

An additional objective is to integrate an AI-powered chatbot that can interact with users in real time. The chatbot will assist users in submitting complaints, provide instant responses, and guide them through the complaint process. This reduces manual effort and enhances user experience by enabling quick and efficient communication.

Another important objective is to support multilingual communication within the system. The project aims to provide responses and summaries in multiple languages, improving accessibility for users from diverse linguistic backgrounds and ensuring inclusivity.

Finally, the project aims to build a scalable and extensible system that can be adapted to different organizational needs. Future enhancements may include explainable AI techniques, voice-based complaint interaction, and predictive analytics to identify recurring issues and improve decision-making processes.

## 1.4 Scope of the Project

The extent of this project involves the development of an AI-powered complaint management system for classifying and managing user complaints efficiently. The system is designed to analyze complaint text and classify them into multiple categories, such as technical issues, service requests, feedback, and general queries. The project covers the full workflow, from preprocessing user inputs to deploying the AI model for automated ticket classification. The system uses a sample dataset of complaints for testing and training purposes. The main focus of this project is on the accuracy and reliability of the classification model, ensuring it effectively supports support teams in decision-making.

The project also involves the development of an advanced deep learning model for natural language understanding. Preprocessing techniques such as text cleaning, tokenization, and augmentation are applied. Training and validation of the model are included within the project scope. Evaluation metrics such as precision, recall, F1 score, and accuracy are used to ensure robust performance. The system also provides confidence scores for transparency in classification.

Another key area covered under the scope is multilingual support for user interactions. The system uses a language model API to provide responses and explanations in English, Hindi, Tamil, and Telugu. This improves accessibility for users with diverse language backgrounds. Responses are made simple, user-centric, and informative. Disclaimers clarify that AI responses are assistive and do not replace professional support, ensuring ethical and responsible usage.

The scope of the project is limited to academic research and prototype development. It does not cover full-scale deployment in enterprise systems or integration with live customer databases. The system is intended as a decision-support tool, not a replacement for human support agents. It does not handle escalations, legal disputes, or advanced analytics beyond complaint classification. Future enhancements may extend functionality to include sentiment analysis, automated resolution suggestions, and predictive support metrics.

Overall, the system demonstrates the integration of deep learning and natural language generation for accessible, AI-assisted customer support. It shows how intelligent complaint management can be implemented using lightweight web frameworks while maintaining reliability, multilingual support, and operational efficiency.

## 1.5 Methodology Overview

The proposed system's methodology begins with data collection and preprocessing using complaint datasets obtained from organizational records or publicly available sources. The dataset consists of textual complaints categorized into multiple classes such as technical issues, billing problems, service requests, and general inquiries. The data is carefully analyzed and divided into training and testing sets. Preprocessing techniques are applied to ensure consistent input formatting, including text cleaning, removal of stop words, tokenization, and normalization. These steps improve data quality and enhance the model's generalization capability.

The processed complaint text is then fed into a transformer-based model for feature extraction and classification. The model converts text into embedded representations and uses self-attention mechanisms to capture both local and global contextual relationships within each complaint. This enables the system to understand user intent, sentiment, and semantic meaning. The transformer architecture allows hierarchical feature learning, improving the system's ability to handle complex and diverse complaint patterns. Extracted features are passed through fully connected layers for final classification, with a Softmax layer generating probability scores for each complaint category. The category with the highest probability is selected as the predicted output, and tickets are automatically assigned and prioritized accordingly.

For performance evaluation, the model is trained using advanced optimization techniques such as the Adam optimizer, along with categorical cross-entropy loss. Training is carried out over multiple epochs with defined batch sizes to ensure optimal learning.

Finally, the classified complaint and its priority level are integrated into an AI chatbot system for real-time interaction. The chatbot assists users in submitting complaints, provides instant responses, and generates user-friendly summaries. The system supports multilingual communication, allowing responses based on user preference. A web-based interface is developed to enable users to submit complaints, track ticket status, and visualize analytics in real time. The end-to-end system ensures automation, accuracy, accessibility, and an enhanced user experience in complaint management.

## 1.6 Organization of the Report

This report is structured to present the research work in a clear and organized manner, allowing readers to follow the development of the project from problem identification to system implementation and evaluation. The chapters are arranged to provide a logical progression of ideas, making it easier to comprehend the objectives, methodology, and results of the project.

Chapter I introduces the background and motivation for developing an AI-powered complaint management system. It presents the problem statement, project objectives, scope, and an overview of the proposed methodology. This chapter highlights the challenges in traditional complaint management processes and emphasizes the need for automation, multilingual support, and intelligent ticket handling. It sets the context for the technical discussions in the subsequent chapters.

Chapter II provides a literature review of existing research and solutions in automated complaint management and AI-powered ticket systems. It covers traditional rule-based systems, machine learning approaches, and transformer-based architectures for text classification and natural language understanding. The chapter analyzes the strengths and limitations of these approaches and identifies research gaps that the proposed system aims to address.

Chapter III explains the proposed methodology, system architecture, and dataset preparation. It describes the preprocessing techniques, transformer-based model design, AI chatbot integration, and multilingual support. This chapter also explains the workflow of the automated complaint management system and how the various components interact.

Chapter IV presents the experimental setup, performance evaluation, and results analysis. Metrics such as accuracy, precision, recall, F1-score, and confusion matrices are used to assess the effectiveness of the system. The chapter also discusses the usability, interpretability, and accessibility features provided by the AI chatbot and web interface.

Chapter V concludes the project by summarizing the major contributions, system capabilities, and potential impact of the solution. It also outlines future enhancements, such as explainable AI integration, voice-based complaint handling, and predictive analytics for recurring issues.

The report concludes with references that acknowledge related research and existing work in the field. The structured presentation ensures that both technical and non-technical audiences can easily understand the concepts, and it facilitates academic evaluation and future research.

This systematic approach makes the project documentation coherent, professional, and accessible for future reference.

## **CHAPTER - II**

### **LITERATURE REVIEW**

#### **2.1 Overview of Related Work**

Automated complaint management and ticketing systems have been a significant area of research in organizational operations and customer service. Early approaches relied on traditional rule-based systems and classical machine learning algorithms such as Support Vector Machines (SVM), Decision Trees, and k-Nearest Neighbors (KNN) for complaint classification [10]. These methods required manual feature extraction from textual data, including keyword-based analysis, term frequency calculations, and pattern matching[16]. While these approaches achieved reasonable results for structured complaints, they often failed to handle complex, unstructured, or context-dependent complaints effectively. Additionally, inconsistencies in user input, language variations, and informal expressions impacted classification accuracy[4].

With the advent of deep learning, neural network-based models, particularly Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks, were increasingly applied for automated complaint classification [5]. These models learn sequential dependencies in text and capture contextual meaning to some extent [20]. They reduced the need for manual feature engineering and improved performance in processing complex complaint data. However, RNN and LSTM models often struggle with long-range dependencies and are computationally intensive for large-scale datasets.

To address these limitations, attention mechanisms were introduced to enhance feature learning by focusing on the most relevant portions of text [12]. Hybrid architectures combining RNNs with attention layers demonstrated improved classification accuracy and better handling of critical complaint features[6]. Some approaches also explored hierarchical text representations to model document-level dependencies. Despite these improvements, many models still faced challenges with multilingual complaints, inconsistent input formats, and real-time response requirements [13].

Recently, transformer-based models have emerged as a state-of-the-art solution for natural language understanding and text classification tasks.

This project builds upon these advancements by employing a transformer-based architecture integrated with an AI chatbot, focusing on automated ticket classification, prioritization, and multilingual user interaction [4]. The proposed system aims to bridge the gap between research-level model performance and real-world usability in complaint management applications [19].

## **2.2 Review of Similar Projects or Research Papers**

Several research studies have explored the use of deep learning models for automated complaint classification and ticket management. Early works primarily utilized traditional machine learning models such as Support Vector Machines (SVM), Decision Trees, and Naive Bayes classifiers [5],[18]. These approaches often relied on manual feature extraction techniques, including bag-of-words representations, TF-IDF vectorization, and keyword matching. While these methods achieved moderate success on structured complaint datasets, they struggled with unstructured and context-rich complaints, leading to limited generalization across different organizations and domains [3].

To address these limitations, recent studies employed deep learning approaches such as Recurrent Neural Networks (RNNs), Long Short-Term Memory (LSTM) networks, and Gated Recurrent Units (GRUs) for text classification [19]. These models automatically learn sequential patterns and contextual dependencies in complaint texts, improving classification performance compared to traditional methods. Data augmentation techniques, including paraphrasing and synonym replacement, were also used to reduce overfitting and improve model robustness. Transfer learning with pre-trained embeddings such as Word2Vec and GloVe further enhanced performance on smaller datasets [6], [7]. However, RNN-based models often struggled with long-range dependencies and were computationally intensive for large-scale complaint datasets.

Hybrid models combining deep learning with classical machine learning techniques have also been proposed. For example, researchers extracted textual features using neural networks and used SVMs or Random Forests for classification. Attention mechanisms were incorporated into LSTM models to focus on critical portions of complaints, improving classification accuracy and interpretability [1]. Some studies also explored hierarchical or segmentation-based approaches to handle multi-part complaints or categorize complaints across multiple departments. While these methods improved accuracy, computational complexity and scalability remained major challenges [21].

## 2.3 Summary and Gap Identification

From the literature review, it is evident that significant progress has been made in automated complaint management and ticket classification using machine learning and deep learning algorithms [10], [21]. Traditional machine learning methods provided initial solutions but heavily depended on manual feature extraction and were limited in handling unstructured and context-rich complaints. The adoption of deep learning models such as RNNs, LSTMs, and attention-based architectures improved automation and contextual understanding [13], [7]. More recently, transformer-based models have demonstrated superior capabilities in capturing long-range dependencies and semantic relationships in textual data, setting a new benchmark for intelligent complaint handling systems [1],[18].

In summary, the existing research highlights advancements in model accuracy and contextual understanding but leaves significant gaps in real-world usability, interpretability, multilingual support, ethical deployment, and real-time performance [22]. The proposed project addresses these gaps by integrating a transformer-based complaint classification system with an AI chatbot, providing multilingual responses, user-friendly interfaces, and efficient real-time ticket management. This approach ensures a holistic solution that balances technical performance, accessibility, and practical usability in organizational complaint management systems [23].

## **CHAPTER - III**

### **SYSTEM ANALYSIS**

#### **3.1 Requirements Gathering**

The requirements gathering phase focuses on identifying the needs of the users and stakeholders for the proposed complaint management system. The target users of the system include support staff, administrators, managers, and general users submitting complaints. The requirements were gathered by analyzing existing complaint management systems, studying research on AI-based ticketing solutions, and consulting with academic mentors to understand real-world challenges. The system is expected to automate complaint classification, ticket prioritization, and real-time user interaction while remaining intuitive and accessible to all users.

Functional requirements were identified by examining similar AI-based text classification and ticketing systems. The system should allow complaint submission in standard text formats through a web interface or chatbot. It should automatically preprocess the textual data before classification, including tokenization, cleaning, and normalization. The system should provide real-time classification and prioritization of complaints, produce probability scores or confidence levels for predictions, and integrate with a multilingual chatbot module for interactive communication.

During the requirements gathering phase, consultations with mentors and references to relevant domain literature were considered to understand user needs and system functionality. Research papers on AI-powered ticketing systems and natural language processing applications were studied to identify common features and best practices. User requirements such as simplicity, ease of use, and quick response times were emphasized. Special attention was given to multilingual support to cater to users from diverse linguistic backgrounds. Real-time system performance was prioritized to enable immediate complaint acknowledgment and ticket generation.

In addition to technical requirements, deployment considerations were incorporated into the requirements analysis. The system should operate efficiently on standard computing resources and be compatible with commonly available web platforms for accessibility. Effective visualization of complaint status, ticket analytics, and classification results was highlighted to improve user understanding. Requirements also included minimal setup complexity for easy

installation and execution, stability under repeated complaint submissions, and robust error handling mechanisms to manage invalid or incomplete inputs effectively.

These requirements form the foundation for designing, implementing, and evaluating a user-friendly, efficient, and practical complaint management system that integrates AI classification with real-time chatbot interaction and multilingual support.

### **3.2 Functional Requirements**

The functional requirements of the proposed complaint management system define the essential tasks that the system must perform to meet user needs effectively. These requirements were gathered by analyzing similar AI-based ticketing systems, studying research on natural language processing applications, and consulting with academic mentors to understand practical expectations.

The system should allow users to submit complaints via a web interface or an AI-powered chatbot. It must support standard text formats and handle both structured and unstructured complaint data. Before classification, the system should automatically preprocess the complaint text by performing tokenization, cleaning, normalization, and removal of irrelevant characters or symbols.

Once preprocessing is complete, the system should classify complaints into predefined categories such as technical issues, service requests, billing issues, or general inquiries. The classification must be accurate and powered by a transformer-based model to capture contextual information from the complaint text. The system should also assign priority levels to complaints based on severity, urgency, or type, ensuring high-priority complaints are flagged for faster resolution.

Real-time processing is a key requirement. The system should provide instant acknowledgment of complaint submissions, real-time classification, and immediate ticket generation. It should also display probability scores or confidence levels to indicate the reliability of the classification results, enabling users and administrators to make informed decisions.

Multilingual support is an important functional requirement. The system should allow users to interact in their preferred language, with the AI chatbot and interface capable of delivering responses in multiple languages. Analytics and visualization features should provide administrators with dashboards and reports for tracking ticket status, complaint trends, and model performance, allowing better monitoring and decision-making.

### 3.3 Non-Functional Requirements

The non-functional requirements define the quality attributes, operational standards, and constraints of the proposed complaint management system. These requirements ensure that the system is efficient, reliable, user-friendly, secure, and scalable for practical deployment.

Performance is an essential non-functional requirement. The AI model should provide accurate complaint classification and ticket prioritization with minimal errors. The time required for preprocessing, classification, and response generation should be short to ensure real-time interaction with users. The system should handle multiple simultaneous user interactions without delays. Resource and memory utilization should be optimized so that the system works efficiently on standard computing infrastructure. Consistent performance is crucial for reliability in real-world usage.

Usability is another key requirement. The user interface should be simple, clean, and intuitive, enabling both technical and non-technical users to submit complaints easily. Instructions for submission should be clearly visible, and outputs such as ticket status, classification results, and priority levels should be presented in a readable and understandable format. Support for multiple languages should improve accessibility, while visual components like dashboards and graphs should enhance interpretability. The system should require minimal technical knowledge for effective operation.

Security and Privacy are critical to ensure trust in the system. Users should be informed that AI-generated classifications do not replace professional advice. Complaint data should not be stored permanently without user consent, and secure communication protocols should be used for all API interactions. Unauthorized access or misuse must be prevented, and ethical standards should be maintained in displaying predictions. Data privacy and responsible AI usage should be enforced throughout the system's operation.

Scalability and Maintainability are necessary for future growth and improvement. The system architecture should allow the integration of new complaint categories or modules, enable updates to more advanced transformer models, and support future incorporation of explainable AI tools. The code should be modular, maintainable, and easy to modify. Compatibility with multiple deployment environments should be ensured to facilitate expansion and future upgrades.

## 3.4 Feasibility Study

### 3.4.1 Technical Feasibility

Technical feasibility assesses the availability of the necessary technology, tools, and infrastructure required to develop the proposed complaint management system. The project leverages Python as the primary programming language, which offers robust support for artificial intelligence and natural language processing applications. Transformer-based models, such as the Swin Transformer, can be developed and fine-tuned using widely adopted deep learning libraries like PyTorch. For text preprocessing, libraries such as NLTK, SpaCy, and Hugging Face Transformers provide the necessary functionality for tokenization, cleaning, normalization, and contextual embedding.

The system architecture supports real-time complaint classification and ticket management. Pretrained transformer models can be fine-tuned to classify complaint texts into categories and assign priority levels effectively. Probability-based outputs can be generated using Softmax layers, and AI-powered chatbots can handle multilingual interactions through secure API integration. Libraries and frameworks like Streamlit or Flask enable efficient web deployment and creation of interactive user interfaces. Response time optimization techniques can be employed to ensure smooth real-time operation.

The development environment further supports collaboration and version control, enabling efficient team development and code management. Libraries for AI model training, text preprocessing, deployment, and web interface creation are well-documented and regularly updated. Cross-validation and other testing strategies can be employed to validate model performance and ensure generalization. The integration of classification, chatbot, and reporting modules is technically feasible, and performance optimization methods can minimize latency, ensuring that the system meets real-time operational requirements.

Overall, the combination of available programming tools, libraries, computing infrastructure, and deployment frameworks confirms that the proposed complaint management system is technically feasible and can be implemented efficiently in an academic or practical environment.

### **3.4.2 Operational Feasibility**

Operational feasibility assesses how effectively the proposed complaint management system can function in real-life situations and meet the needs of its intended users. The system is designed with a simple and intuitive web interface, allowing users to submit complaints and interact with the AI-powered chatbot with minimal guidance. Users can choose their preferred language, ensuring accessibility for a linguistically diverse audience. The system provides clear outputs, including complaint category, priority level, and confidence scores, along with visual indicators to reduce confusion and enhance interpretability. Real-time processing allows quick acknowledgment and classification of complaints, making the system practical for day-to-day operations.

The system is user-friendly and requires minimal training for effective operation. Both technical and non-technical users, including support staff, administrators, and general users, can interact with the system easily. The AI-based classification and automated ticket generation reduce manual workload and help streamline complaint management. Multilingual support ensures that users from different regions can communicate effectively, and clear disclaimers maintain transparency about AI-generated results. These features make the system suitable for deployment in various environments, such as corporate offices, service centers, and research institutions.

Maintenance and usability have been carefully considered in the operational analysis. The system does not involve complicated installation processes, and updates to the AI model or system modules can be performed without redesigning the entire application. Error messages and feedback mechanisms are designed to guide users appropriately during interactions. Multiple complaint submissions can be handled within a single session, and normal usage does not affect system performance. The application serves as an effective decision-support tool, assisting staff in managing complaints efficiently.

The web-based architecture ensures accessibility from any device with internet connectivity, eliminating the need for specialized hardware. The system provides visual feedback during data processing and prediction tasks, enhancing user confidence. It is capable of functioning reliably on moderate computational resources, making it feasible for deployment in offices or institutions with standard IT infrastructure. The simple workflow—from complaint submission.

### 3.4.3 Economic Feasibility

Economic feasibility evaluates the cost-effectiveness of developing, deploying, and maintaining the proposed complaint management system. The project primarily relies on open-source software and tools, eliminating the need for expensive proprietary solutions. Python, PyTorch, Hugging Face Transformers, NLTK, and Streamlit are all freely available, providing a robust development environment at minimal cost. The dataset used for model training and evaluation is publicly accessible, further reducing initial investment requirements. Additionally, the project can be developed and tested using existing academic or institutional computing infrastructure, making the initial development expenses significantly low.

Deployment costs are also minimal due to the use of lightweight web frameworks such as Streamlit, which allow the system to run on standard local machines without requiring costly infrastructure. Cloud deployment is possible if needed, with scalable pricing structures that enable cost management based on actual usage. The modular structure of the code ensures that maintenance and future enhancements can be performed efficiently without significant redevelopment costs. No specialized hardware is required beyond basic GPU support for model training, further reducing overall expenditure.

From a long-term perspective, the system offers high functional value relative to its cost. It serves as a research prototype and academic demonstration system while providing practical features such as real-time complaint classification, ticket prioritization, and multilingual chatbot integration. The multilingual summary generation is implemented using open-source libraries and APIs, avoiding additional licensing or subscription costs. This ensures that the cost-benefit ratio remains highly favorable for academic, research, and low-resource institutional use.

Future upgrades and enhancements can be performed incrementally due to the modular and well-documented codebase. Maintenance requirements are minimal, and incremental improvements can be integrated without incurring significant financial burden. The inclusion of multilingual support, real-time processing, and AI-powered classification enhances the system's functional value without increasing costs.

Overall, the reliance on open-source technology, minimal infrastructure requirements, low deployment and maintenance costs, and high long-term utility confirm that the proposed

complaint management system is economically feasible, sustainable, and cost-effective for both academic and practical applications.

### **3.5 Risk Analysis**

Risk analysis is a critical step in identifying potential factors that might impact the performance, reliability, and usability of the proposed complaint management system. Addressing these risks proactively ensures that the system operates effectively and safely in real-world conditions.

Technical Risks are among the primary concerns. One significant risk is model overfitting due to limited or non-diverse training data. If the dataset does not adequately represent real-world variations, the model's prediction accuracy may decrease. Misclassification of rare complaint types is another potential risk. Limitations in computational resources could slow down model training and real-time prediction performance. Additionally, reliance on external APIs for multilingual summary generation may cause delays or inconsistencies in response.

Data quality is also a key technical risk. Poor-quality input data, inconsistencies in preprocessing, or incorrect normalization can lead to loss of critical information and reduce classification accuracy. Imbalanced datasets may introduce bias in predictions, favoring more frequently occurring complaint types. Hardware failures, insufficient memory, or network disruptions can interrupt model training, real-time classification, or API communication, affecting overall system reliability.

Operational Risks must also be carefully considered. Users may interpret AI-generated predictions as final or authoritative results without professional verification, potentially leading to misuse or misinterpretation. Ethical concerns may arise if system limitations are not clearly communicated. Data privacy is another critical consideration; uploaded data must be protected against unauthorized access or misuse. Clear disclaimers and user guidelines are essential to mitigate these operational and ethical risks.

Preventive Measures are recommended to reduce the likelihood and impact of these risks. Cross-validation, rigorous performance evaluation, and monitoring can help prevent overfitting and ensure robust model performance. Expanding the dataset and incorporating diverse examples will improve generalization. Computational optimization methods can reduce latency

and resource usage. Secure communication protocols should be implemented for API interactions. Clear guidelines, disclaimers, and usage instructions must be provided alongside predictions to ensure ethical and responsible use.

By addressing these technical and operational risks with preventive measures, the system can be deployed safely, reliably, and effectively, ensuring trustworthiness and practical applicability for real-world use.

***Note:** The risk analysis process, drawing on concepts from 212CSE2304 – Machine Learning and 213CSE3301 – Deep Learning, identifies potential factors that may affect system performance and accuracy. The system faces technical challenges such as limited dataset diversity, which may lead to overfitting, difficulties in accurately classifying rare tumor types, and computational limitations during model training. Furthermore, classification performance can be influenced by MRI image quality, improper preprocessing, and dataset imbalance, which may affect the reliability and generalization of the model.*

## **CHAPTER - IV**

### **SYSTEM DESIGN**

#### **4.1 Overall System Architecture**

The system architecture represents the complete workflow of the proposed Complaint Care system, highlighting how AI-driven components interact to manage complaints efficiently. The workflow begins with a user submitting a complaint through the web interface or AI chatbot. The complaint text is transmitted to the preprocessing module, which performs tokenization, text cleaning, normalization, and other necessary transformations to prepare the data for analysis. The preprocessed text is then passed to the Transformer-based classification model. The model extracts contextual features using self-attention mechanisms, capturing both local and global dependencies in the complaint text. It predicts the complaint category and assigns a priority level. The highest probability score determines the final classification, and the ticket is automatically generated with relevant details.

The prediction results are sent to the multilingual summary module, which generates a user-friendly explanation in multiple languages, improving accessibility for users from diverse linguistic backgrounds. Finally, the complaint status, ticket ID, priority level, and explanatory summary are displayed through the web interface or returned via the AI chatbot. The architecture follows a modular and layered design approach for scalability and maintainability. The frontend, built using a web framework and chatbot interface, ensures an interactive and intuitive user experience. The backend handles text preprocessing, model inference, ticket generation, and summary creation, with sequential and organized communication between modules.

#### **4.2 Module Design**

The proposed Complaint Care system is organized into multiple functional modules, each designed to perform a specific role within the overall workflow. This modular approach simplifies development, testing, and maintenance while ensuring a seamless flow of data across the system. Each module communicates with others through clearly defined interfaces and standardized data formats, making the system reliable and scalable. Errors in one module do not propagate to others, and future upgrades or additional features can be incorporated without impacting the overall system. Security measures are included in relevant modules to safeguard sensitive user information.

The complaint submission module serves as the entry point for users. Complaints can be submitted via the web interface or AI chatbot. This module validates the input to ensure completeness and accuracy before passing it on for processing. The preprocessing module prepares the complaint text for AI-based classification. It performs tasks such as text cleaning, tokenization, normalization, and removal of unnecessary noise, ensuring that the data is in a consistent format suitable for model inference.

Finally, the security and logging module ensures secure handling of user data, prevents unauthorized access, and maintains detailed logs for monitoring system usage and performance.

#### **4.2.1 Module 1 – Image Upload and Preprocessing**

The responsibility of the Complaint Submission and Preprocessing Module is to receive complaints from users and prepare them for analysis. Users can submit complaints through the system's web interface or AI chatbot in standard text formats such as plain text or JSON. The system validates the submitted complaints, ensuring that the text is complete, in the correct format, and free from unsupported characters.

After validation, the complaints are preprocessed to make them suitable for AI-based classification. Preprocessing includes tasks such as cleaning the text, tokenization, normalization, and removal of unnecessary noise or special characters. This ensures that the complaint text is uniform and compatible with the transformer-based model. The module also performs checks to prevent corrupted or incomplete data from being processed. Proper preprocessing improves the quality and clarity of the input, enabling accurate classification and prioritization of complaints by the system.

#### **4.2.2 Module 2 – Classification and Summary Generation**

The Classification and Summary Generation Module is the core part of the system, responsible for categorizing complaints and generating user-friendly summaries. The module uses a transformer-based architecture to analyze the complaint text and extract relevant features, capturing both local and global contextual information. Feature extraction allows the system to identify key patterns in the text, enabling accurate classification of complaints into predefined categories and assigning appropriate priority levels.

Once the features are extracted, the classification process determines the complaint type and priority. Probability scores or confidence levels are generated for each prediction, allowing the

system to choose the most likely category and priority for the complaint. The system also logs all predictions for future reference, analysis, and model improvement.

The summary generation component produces clear and understandable feedback for users. Multilingual support allows the summaries to be generated in multiple languages, ensuring accessibility for a diverse user base. Visualization components can be extended to include charts or graphs showing complaint statistics or classification confidence for improved interpretability.

### **4.2.3 Module 3 – Multilingual Summary Generation Module**

The Multilingual Summary Generation Module provides a clear and understandable description of the complaint classification results. Once a complaint has been analyzed and classified by the system, this module generates a concise summary explaining the classification and priority of the complaint. The summary is presented to the user in a simple and informative manner, making it easy to understand for both technical and non-technical users.

The system uses an API to translate and display the summaries in multiple languages, enhancing accessibility for users from diverse linguistic backgrounds. The generated summaries include information about the complaint category, assigned priority, and suggested actions or status updates. This ensures that users receive a comprehensive understanding of their complaint without needing specialized knowledge.

### **4.2.4 Module 4 – Result Visualization and User Interface Module**

The Result Visualization and User Interface Module is responsible for presenting the final classification results and related details to the user in an organized manner. Once a complaint is analyzed and classified, the system displays the complaint category, priority level, and confidence scores generated by the model. Graphical representations, such as probability distribution charts, are used to illustrate the model's confidence for each classification, helping users understand the certainty of the predictions.

The user interface is designed to be simple, intuitive, and easy to navigate, ensuring a smooth experience for users of varying technical expertise. Error messages are provided to guide users in case of incorrect or incomplete input. Interactive visual elements facilitate clear interpretation of results and allow users to explore complaint details effectively. Feedback mechanisms can also be included to gather user insights on system performance and usability.

This module emphasizes clear presentation and easy navigation across different system functionalities. The layout and design of the interface enhance user understanding of the classification results and provide a seamless interaction between the user and the system. By integrating visualization, interactivity, and usability features, this module ensures that users can efficiently access, comprehend, and act upon the information provided by the complaint management system.

### **4.3 Database Design**

The database design determines the organization and management of system data in the context of the complaint management application. While the main purpose of the system is to classify complaints and manage tickets in real time, the use of a database facilitates record-keeping, tracking, and analysis at later stages. The database stores user data, complaint details, ticket information, classification results, priority levels, chatbot interactions, and multilingual summary outputs. Structured tables are designed to ensure data consistency, prevent duplication, and maintain logical relationships among different data entities. Primary keys uniquely identify each record for efficient retrieval, while foreign keys link related records, such as users and their submitted complaints. Normalization rules are applied to avoid redundant data and optimize storage efficiency. Security measures such as access controls and encryption can be implemented to protect sensitive user information. Backup and recovery mechanisms ensure data reliability in case of system failures. The database design improves the overall management, traceability, and scalability of the system.

The structure of the database also supports analytics and reporting functions. Historical complaint records can be analyzed to monitor trends in complaint categories, ticket resolution times, and system performance. Indexing enhances the speed of query execution and optimizes data retrieval. Access controls prevent unauthorized access or modification of stored data.

#### **4.3.1 ER Design**

The proposed Complaint Care System uses a database to manage complaint records, ticket information, user details, and classification results. A conceptual Entity-Relationship (ER) model helps to illustrate the relationships among the main entities involved in the system. The primary entities include User, Complaint, Ticket, Classification, Priority, Chatbot Interaction, and Multilingual Summary.

The Classification entity stores the results of the AI-based complaint classification process, such as predicted category and confidence score. Each complaint is linked to a classification record, establishing a one-to-one relationship. The Priority entity represents the urgency level of each ticket, determined by the classification module, and is associated with the Ticket entity. The Chatbot Interaction entity stores user interactions with the chatbot, including query and response pairs, which can help improve system performance. The Multilingual Summary entity stores translated versions of complaint summaries generated for user accessibility, linked to the Classification entity.

### **4.3.2 Database Schema**

The proposed Complaint Care System does not implement a conventional database; hence, the design of a database schema, including tables, primary keys, and foreign keys, is not applicable. The system is intended for real-time complaint processing, and all data handling occurs temporarily during runtime without persistent storage. User-submitted complaints are processed immediately by the preprocessing and classification modules, and the results, including ticket prioritization and multilingual summaries, are presented in real time through the user interface. No complaint records, user details, or chatbot interactions are stored after the session ends. The system follows a stateless design paradigm, where all data is valid only during the active session and is deleted once the session concludes. This approach ensures privacy, reduces storage requirements, and simplifies the overall system architecture. Consequently, relational tables, structured schemas, or long-term data storage are not defined or required for this project.

## **4.4 User Interface Design**

The User Interface of the proposed Complaint Care System is developed using the Streamlit framework to provide a simple, interactive, and user-friendly environment. The interface is designed to allow users to submit complaints easily, select their preferred language, and view real-time ticket classification and prioritization results. The system ensures smooth navigation with minimal steps, making it accessible to both technical and non-technical users. Once a complaint is submitted, the predicted category, priority level, and confidence score are clearly displayed. Additionally, a multilingual summary generated using an API is presented along with an AI-based disclaimer. The design of the system focuses on clarity, responsiveness, accessibility, and ease of understanding to enhance the overall user experience. The dashboard of the system includes the following components:

- Complaint Text Input Section
- Complaint Preview Display
- Submit Button
- Complaint Category Prediction Display
- Priority Level Display
- Confidence Score Display
- Probability Distribution Bar Chart for Classification
- Multilingual Summary of Complaint and Priority
- AI Disclaimer Message
- Clean and Minimal Layout for Easy Navigation

#### 4.4.1 User Flow Diagram

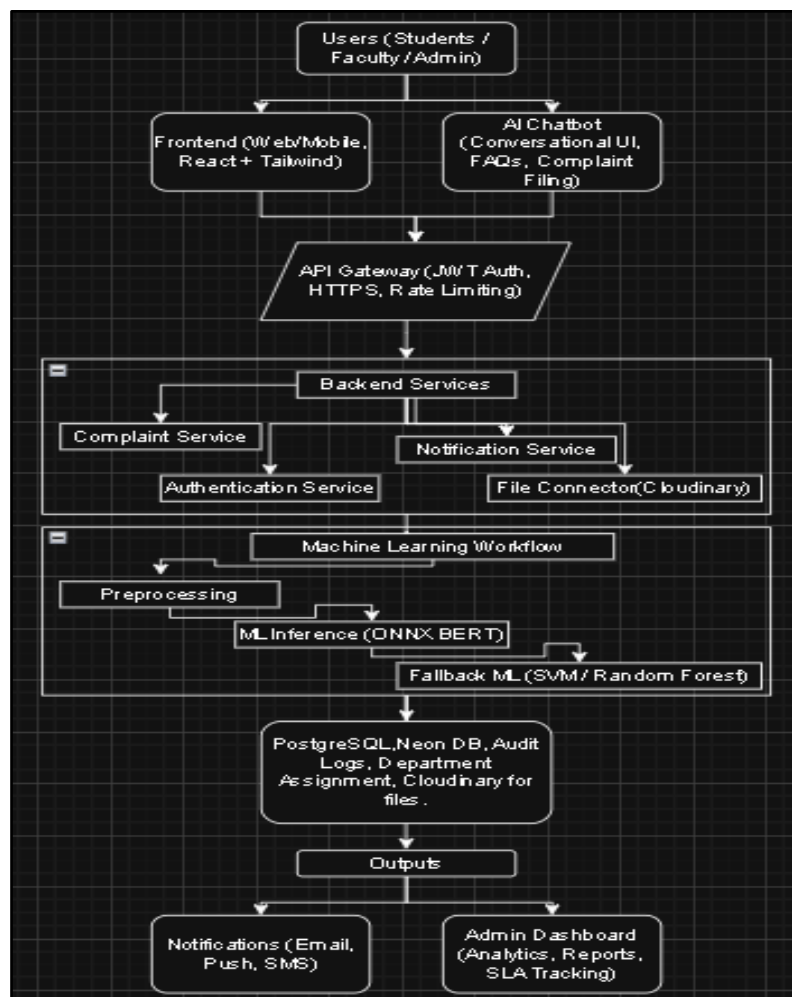


Fig 1. User Flow Diagram

The above diagram shows the overall workflow of the proposed Complaint Management System. The system starts with the selection of the preferred language for the complaint summary and the input of the complaint text by the user. The entered complaint will then go through preprocessing steps such as text cleaning, tokenization, and normalization. The preprocessed complaint text will then be fed to the AI classification model, which will perform the classification of the complaint and provide the predicted complaint category along with the priority level and confidence scores.

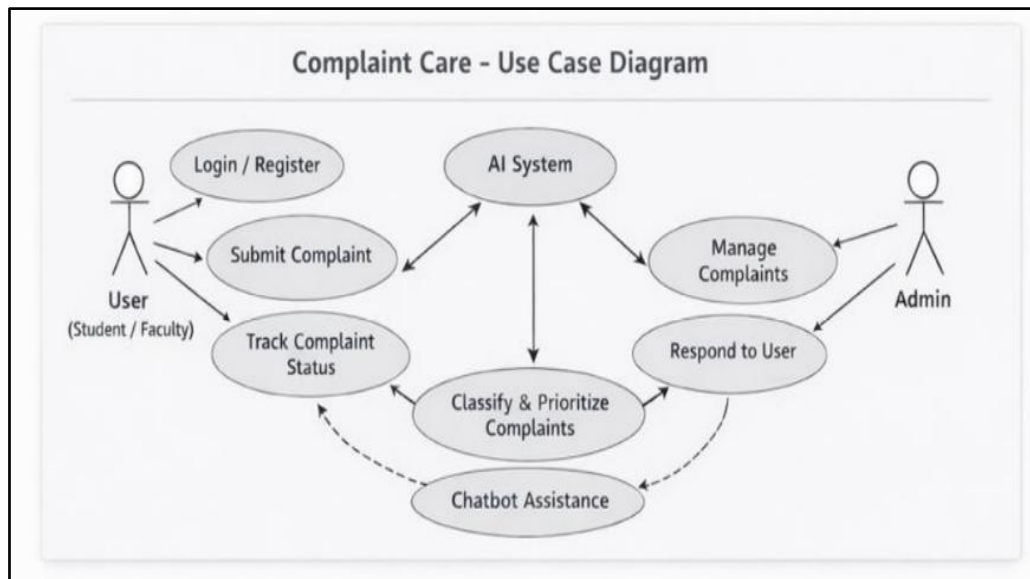


Fig 2. Use Case Diagram

The diagram illustrates the functioning of a complaint care system where users such as students or faculty interact with the platform to register, submit complaints, and track their status. Once a complaint is submitted, it is handled by an AI system that acts as the central component, analyzing each complaint by classifying it into categories and assigning priority levels based on urgency. The system also includes chatbot assistance to provide immediate support to users, guide them through the process, and handle simple queries efficiently. On the administrative side, admins manage the complaints by reviewing them, taking appropriate actions, and responding to users with updates or resolutions. Overall, the system ensures a smooth flow from complaint submission to resolution by integrating user interaction, intelligent processing, and administrative control.

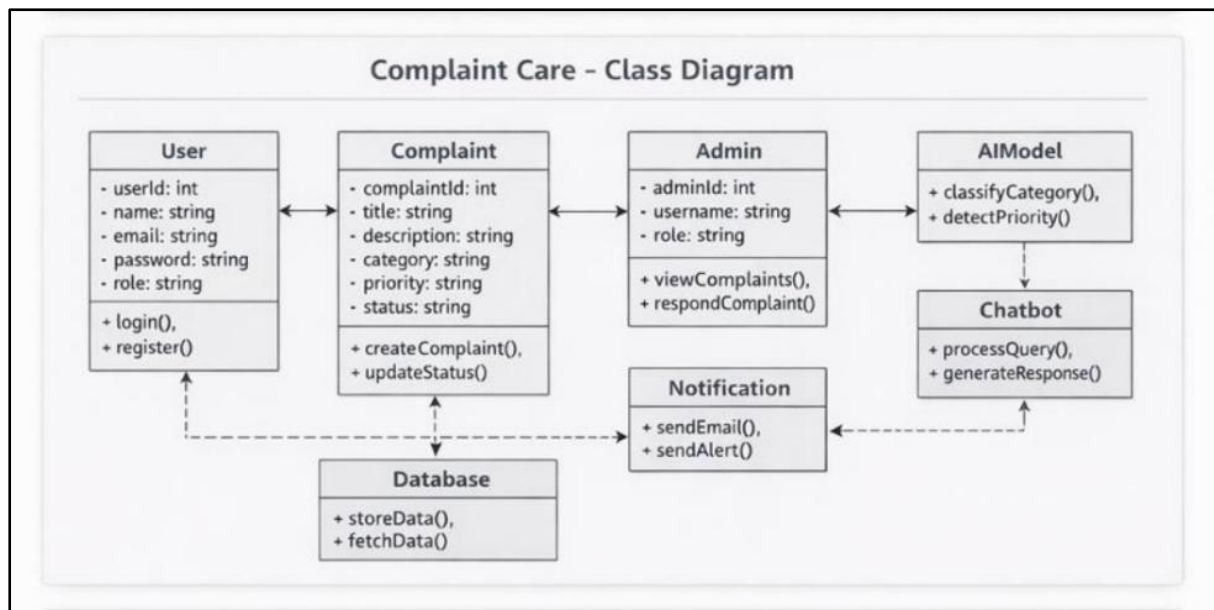


Fig 3. Class Diagram

The diagram represents the class structure of a complaint care system, showing how different components interact within the application. It includes classes such as user, complaint, admin, AI model, chatbot, notification, and database. The user class contains attributes like `userId`, `name`, `email`, `password`, and `role`, along with methods for `login` and `registration`. The complaint class stores details such as `complaintId`, `title`, `description`, `category`, `priority`, and `status`, and includes methods to `create` and `update` complaints. The admin class manages the system with attributes like `adminId`, `username`, and `role`, and provides functionalities to `view` and `respond` to complaints. The AI model class is responsible for intelligent processing by `classifying` complaint categories and `detecting` their priority, while the chatbot class supports user interaction by `processing` queries and `generating` responses. The notification class handles communication through email and alerts, ensuring users receive updates. The database class is used for storing and retrieving all system data. Relationships between these classes indicate that users create complaints, admins manage them, the AI model and chatbot assist in processing, notifications keep users informed, and all data is maintained within the database, forming a complete and interconnected system.

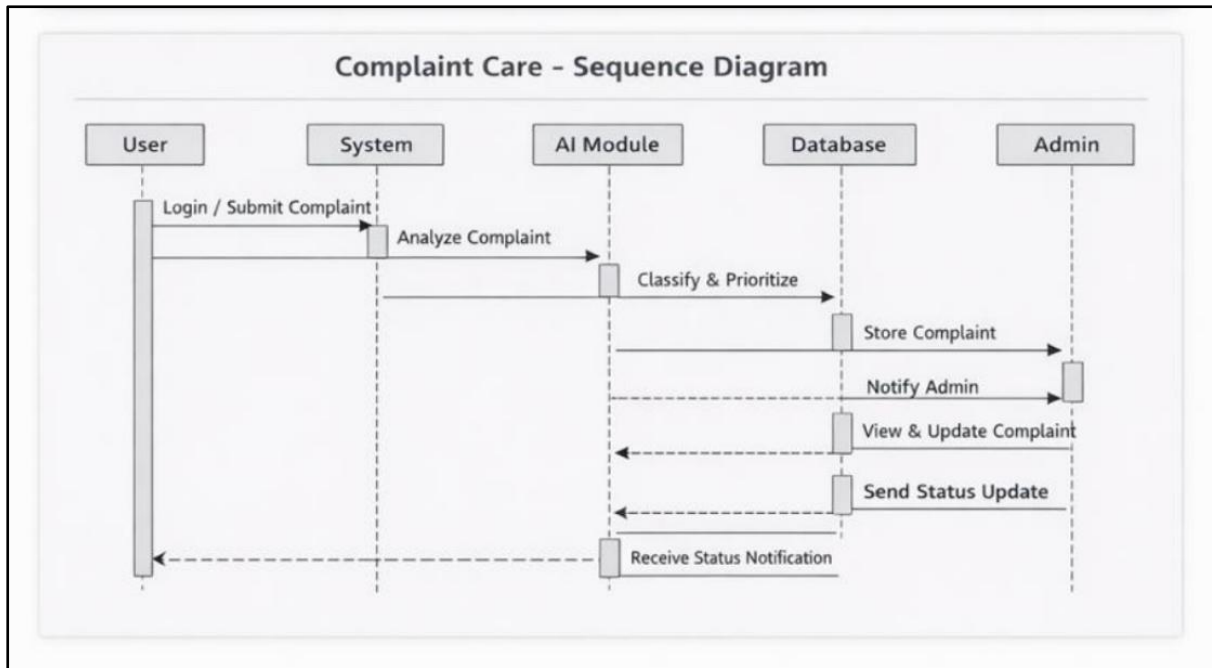


Fig 4. Sequence Diagram

A sequence diagram for the complaint care system illustrates the step-by-step interaction between different components over time when a user raises and resolves a complaint. The process begins when the user logs into the system and submits a complaint with relevant details. This request is sent to the system, where the complaint is stored in the database. The AI model then processes the complaint by classifying its category and assigning a priority level based on its urgency. Simultaneously, the chatbot may assist the user by acknowledging the complaint or providing immediate responses for simple queries.

**Note:** The proposed Complaint Management System, developed using concepts from 211CSE1402 – Python Programming, 213CSE3301 – Web Development, and 213CSE3303 – Natural Language Processing, does not employ a conventional database; hence, the ER design is conceptual and represents functional relationships among entities such as User, Complaint, Category Selection, Status Update, Response, and Final Resolution. The system processes complaints in real-time using automated workflow logic and displays status updates along with AI-generated responses via a Streamlit interface.

## **CHAPTER - V**

### **IMPLEMENTATION**

#### **5.1 Technology Stack**

The proposed Complaint Management System is developed with a strong and flexible technology stack suitable for real-time complaint handling and automated workflow management. The system is primarily developed using Python due to its simplicity, flexibility, and strong support for web development, automation, and natural language processing. Python's extensive ecosystem allows seamless integration of NLP libraries, web frameworks, and AI modules. Its modularity supports efficient development, testing, and maintenance of individual system components. Python is also compatible with multiple operating systems, making the deployment process smooth across different platforms. In summary, Python serves as the foundation for the entire system development.

For natural language processing and intelligent complaint response generation, the system uses libraries like spaCy and NLTK, which enable efficient text preprocessing, entity recognition, and sentiment analysis. Additionally, the transformers library is integrated for generating AI-assisted responses to user complaints. These libraries allow the system to analyze complaint text, categorize issues, and generate automated but contextually appropriate responses. GPU acceleration can be utilized for faster processing of large text datasets or advanced NLP models.

The web-based user interface is developed using Streamlit, an interactive web application framework that enables rapid deployment of Python-based systems. Streamlit provides pre-built widgets such as file uploaders, drop-down boxes, buttons, and charts, which are used to manage complaints, track status, and visualize complaint statistics. For multilingual complaint responses and summaries, the system incorporates the Gemini API, allowing the generation of user-friendly messages in English, Hindi, Tamil, and Telugu.

The system also uses Python libraries like Pandas and OpenPyXL for handling complaint data in different formats (CSV, Excel). These libraries help in reading, organizing, and preprocessing user complaint information before analysis or display. Effective data handling ensures that complaint tracking, categorization, and reporting are accurate and efficient.

The development and deployment environment is designed to be flexible and scalable. While the system can run efficiently on standard computers for day-to-day operations, heavy NLP

processing or large-scale batch processing can be accelerated using GPU-enabled platforms. The code is structured modularly, making it easy to debug, maintain, and expand with additional features, such as analytics dashboards, advanced reporting, or AI-driven sentiment insights. The implementation ensures that the system is efficient, maintainable, and scalable for future enhancements.

### **5.1.1 Programming Languages and Tools**

The proposed Complaint Management System implementation is done using Python as the main programming language because of its simplicity, flexibility, and extensive support for web development, automation, and natural language processing. Python allows for easy integration of NLP libraries, data handling tools, and web application frameworks in one environment.

For intelligent complaint analysis and response generation, the system uses NLP libraries such as spaCy, NLTK, and transformers, which support text preprocessing, categorization, and AI-driven response generation. The web-based user interface is created using Streamlit, enabling real-time interaction, complaint submission, status tracking, and visualization without requiring complex backend programming. For multilingual complaint summaries and automated responses, the Gemini API is incorporated to provide user-friendly outputs.

Data handling and preprocessing for complaints are managed using Pandas and OpenPyXL, which allow efficient reading, organization, and analysis of structured complaint data. Development and testing of the system were done using platforms such as Visual Studio Code.

The combination of these programming languages, libraries, and tools ensures efficient implementation, scalability, and real-time system performance.

- Programming Languages: JavaScript, HTML, CSS
- Front-End Framework: React.js
- Back-End Framework: Node.js, Express.js
- Database: MongoDB
- API Development & Integration: RESTful APIs, Axios
- State Management: Redux (optional for larger projects)
- Visualization Tools: Chart.js, Recharts
- Development Platforms & Tools: VS Code, Postman, Git

## **5.2 Implementation of Modules**

The proposed Complaint Management System is developed using a modular structure to ensure clarity, scalability, and maintainability. Each functional part of the system is designed and developed as a separate module that handles a specific task in the overall workflow. The system implementation begins with the front-end user interface module using React.js for complaint submission, tracking, and visualization. The back-end API module is built with Node.js and Express.js to handle complaint processing, categorization, and status updates. The database module uses MongoDB to store complaint details, user information, and resolution history.

The modular design enables seamless communication between modules via RESTful APIs, ensuring organized data flow and easy maintenance. For example, the complaint categorization logic can be enhanced with advanced NLP models in the future, and additional features like multilingual automated responses can be integrated without affecting other modules. The system is designed to handle real-time complaint management efficiently, providing quick updates and maintaining data integrity, while ensuring scalability for future enhancements.

### **5.2.1 Module 1: User Interaction and Preprocessing Module**

The User Interaction and Preprocessing Module is responsible for handling all front-end tasks and preparing user complaint data for processing. This module is implemented using React.js to create an interactive and user-friendly interface. The module enables users to submit complaints, select complaint categories, and optionally choose their preferred language for communication. After submitting a complaint, the system validates the input data, shows a preview of the submitted information, and begins preprocessing tasks.

The preprocessing tasks involve checking for required fields, sanitizing user input, and structuring the complaint data into a format suitable for the back-end API and database. These tasks ensure that the complaint information is consistent, complete, and ready for processing by the categorization and response modules. This module also handles temporary session data during runtime, enabling users to interact smoothly with the interface without storing unnecessary personal information. Overall, the module ensures a smooth user interface, proper complaint data formatting, and real-time data processing for an efficient complaint management workflow.

### **5.2.2 Module 2: Classification and Multilingual Summary Module**

The Complaint Categorization and Response Module is responsible for analyzing submitted complaints, categorizing them, and generating automated responses. In this module, the preprocessed complaint data from the front-end is sent to the back-end API, where it is processed using NLP techniques and models such as spaCy, NLTK, or transformers. The system categorizes each complaint into predefined classes (e.g., billing issue, service request, technical problem, or general feedback) and assigns a confidence score indicating the likelihood of the classification.

Following categorization, the predicted category and relevant complaint details are used to generate a user-friendly response. If multilingual support is selected, the Gemini API generates the response in the chosen language along with standard disclaimers or instructions as needed. The final output provided to the user includes the complaint category, confidence score, response message, and optionally visualizations such as progress tracking or status updates. This module ensures accurate complaint classification, efficient automated responses, and seamless multilingual support for enhanced user experience.

### **5.3 Integration of Modules**

The integration of the modules in the proposed Complaint Management System uses a structured and sequential pipeline architecture to ensure smooth data flow, real-time processing, and efficient complaint management. The system starts with the User Interaction and Preprocessing Module, where the user submits a complaint, selects a category, and optionally chooses a preferred language for communication. The submitted complaint is immediately validated and preprocessed without storing unnecessary personal information. Preprocessing tasks include checking for required fields, sanitizing input, and structuring the complaint data for backend processing. The smooth transition from user interaction to preprocessing ensures that all complaints are standardized and ready for categorization.

The preprocessed complaint data is then passed to the Complaint Categorization and Response Module. In this module, NLP techniques and models such as spaCy, NLTK, or transformers are used to classify complaints into predefined categories (e.g., billing issue, technical problem, service request, or general feedback). A confidence score is calculated for each categorization, indicating the reliability of the classification. The output is then formatted for generating responses and updates. Programmatic integration between preprocessing and categorization

ensures that complaint data is immediately available for the next stage. This real-time flow enhances system responsiveness and efficiency.

**Note:** The Complaint Categorization and Multilingual Response Module, developed using concepts from 213CSE3301 – Web Development and 213CSE3303 – Natural Language Processing, is responsible for complaint classification and automated response generation. The preprocessed complaint data is input to an NLP model (e.g., spaCy or transformers) for feature extraction and categorization into classes such as billing issue, service request, technical problem, or general feedback. The final category is selected based on the highest confidence score, and the result is passed to the Gemini API to generate a multilingual response in the user-selected language.

## CHAPTER - VI

### TESTING

#### 6.1 Testing Methodology

The proposed Complaint Management System is developed as a full-stack web application using the MERN stack, consisting of MongoDB, Express.js, React.js, and Node.js. The system is designed to manage complaints efficiently, provide real-time updates, and generate multilingual automated responses, making it suitable for diverse user needs. The architecture follows a modular approach, ensuring clarity, maintainability, and scalability. Each module is developed to handle a specific functionality while interacting seamlessly with other components of the system.

The implementation begins with the User Interaction and Preprocessing Module, which is responsible for managing all front-end operations. Built using React.js, this module provides a responsive and intuitive interface where users can submit complaints, choose categories, and optionally select a preferred language such as English, Hindi, Tamil, or Telugu. Once a complaint is submitted, the module performs preprocessing tasks including input validation, sanitization, and structuring of the data into a standard format that is compatible with the backend processing. Temporary session data is preserved during runtime to maintain smooth interactions without storing sensitive information unnecessarily. This ensures that the system is user-friendly, maintains privacy, and prepares the data for efficient processing.

After preprocessing, the complaint data is forwarded to the Complaint Categorization and Response Module. This module utilizes natural language processing techniques and AI models such as spaCy, NLTK, and transformers to analyze the content of the complaint and classify it into predefined categories, including billing issues, service requests, technical problems, or general feedback. The module calculates a confidence score for each categorization, which reflects the reliability of the classification. Based on this output, the system generates an automated response to the user. If multilingual support is selected, the Gemini API is employed to create responses in the chosen language, accompanied by standard instructions or disclaimers. The modular design allows for future enhancements, such as adding new categories, improving classification models, or supporting additional languages without impacting other components.

The Database and Real-Time Update Module ensures that all complaints, their categories, and responses are securely stored in MongoDB for tracking and analytics purposes. This module maintains the integrity of complaint records and supports real-time updates to the user interface. By leveraging RESTful APIs built with Express.js and Node.js, the front-end and back-end communicate seamlessly, enabling users to view the status of their complaints immediately after submission or after any update from the system administrator.

The integration of these modules follows a sequential pipeline architecture, ensuring smooth data flow and real-time processing. Complaints are first submitted and preprocessed, then categorized, and finally responded to with optional multilingual support. The system guarantees that all information is available instantly to the user interface, maintaining responsiveness and reliability. The modular and integrated approach allows each component to be enhanced or replaced independently, providing scalability and flexibility for future improvements.

### **6.1.1 Unit Testing**

Unit testing is carried out to verify the functionality of individual modules of the Complaint Management System before they are integrated into the complete system. Each module, including the User Interaction and Preprocessing Module, Complaint Categorization and Response Module, and Database and Real-Time Update Module, is tested separately to ensure that all components perform their intended tasks correctly.

The user interface module is tested for proper input validation, ensuring that complaints are submitted with all required fields and in supported formats. The preprocessing module is tested to confirm that input data is sanitized, structured, and formatted correctly before being passed to the backend for categorization. The categorization module is tested to ensure that complaints are accurately classified into predefined categories and that confidence scores are calculated correctly. The response module is tested to verify that automated responses are generated properly, including multilingual outputs via the Gemini API, and that API communication functions as expected.

Test cases are performed using both valid and invalid inputs to evaluate the system's error-handling capabilities. Boundary conditions, such as unusually long complaint descriptions, missing fields, or unsupported input formats, are also tested. This helps ensure that the system remains stable and responsive under all expected and unexpected conditions.

### **6.1.2 Integration Testing**

Integration testing is carried out after the successful completion of unit testing to evaluate the functionality of the Complaint Management System as a whole, ensuring that the individual modules interact seamlessly. The primary objective is to verify the smooth flow of data across interconnected modules, including the User Interaction and Preprocessing Module, the Complaint Categorization and Response Module, the Database and Real-Time Update Module, and the Automated Multilingual Response Module. During integration testing, the functionality of the front-end complaint submission interface and preprocessing of user inputs is thoroughly verified. The structured complaint data is then passed to the categorization module, and the accuracy and consistency of category assignments and confidence scores are validated. The integration of the categorization results with the response generation module is tested to confirm that the system produces appropriate automated replies, including multilingual outputs through the Gemini API. The interaction between the front-end and back-end modules is examined rigorously to ensure seamless communication via RESTful APIs.

### **6.1.3 System Testing**

System testing is performed to evaluate the fully integrated Complaint Management System, ensuring that all functional and non-functional requirements are satisfied. The primary goal of system testing is to validate that the system works correctly in a real-time environment and meets the expectations for performance, reliability, and usability. The testing process examines the entire workflow, from complaint submission through preprocessing, categorization, automated response generation, and final display of results on the user interface.

Functional testing ensures that each module operates as intended according to the design specifications. This includes verifying the proper functioning of complaint submission forms, data preprocessing routines, complaint categorization accuracy, automated response generation, and multilingual support via the Gemini API. Non-functional testing evaluates performance aspects such as response time, system reliability, stability, and scalability under different load conditions. Compatibility testing is performed to ensure that the system works seamlessly across various browsers, devices, and screen sizes, while security testing confirms that sensitive complaint data is handled appropriately and protected from unauthorized access.

System testing is carried out using real-time scenarios and a variety of complaint types to simulate actual usage conditions. Stress testing is performed to understand how the system

behaves under high traffic or multiple concurrent complaint submissions. Metrics such as response time for complaint processing, categorization accuracy, and automated response generation are measured precisely. The correctness of the output, including complaint categorization, confidence scores, and multilingual responses, is verified against expected results. Error handling mechanisms are also tested to ensure that the system recovers gracefully from invalid inputs or unexpected events. Any identified issues are documented and corrected, followed by regression testing to confirm that the fixes maintain system stability.

#### **6.1.4 User Acceptance Testing (UAT)**

User Acceptance Testing (UAT) is conducted to evaluate the Complaint Management System from the end-user's perspective before its final deployment. The primary objective of UAT is to ensure that the system meets the expectations of the users and is practical for real-world use. Selected users, such as students, office staff, and administrators, interact with the system in a controlled environment, performing the complete workflow from complaint submission to categorization, automated response generation, and status tracking.

During UAT, users assess the system interface for ease of use and intuitiveness. They verify whether complaint categories, status updates, and confidence indicators are displayed clearly. The multilingual response functionality is also tested to ensure that messages are readable, contextually accurate, and useful in languages such as English, Hindi, Tamil, and Telugu. Users evaluate system responsiveness and overall usability, providing feedback on any areas where the interface or workflow could be improved.

Real-life scenarios are simulated during UAT to observe user behavior and interaction patterns. Users are encouraged to provide inputs that can help enhance system functionality, improve design, or clarify instructions. Any ambiguity in the user interface, output messages, or response formatting is documented. The system is checked for consistency in performance during repeated interactions, and all identified issues or suggestions are analyzed thoroughly. Based on this feedback, necessary modifications are implemented, and the system is retested to ensure that user concerns have been addressed satisfactorily.

UAT also evaluates the overall usefulness and reliability of the system. Users confirm that complaint categories, automated responses, and status updates are presented clearly and accurately. Graphical representations, such as charts for complaint statistics, are verified for correctness and clarity. Multilingual outputs are assessed for clarity and ease of understanding.

Any performance delays, minor bugs, or interface difficulties encountered during usage are recorded, and recommendations for enhancements or new features are considered carefully.

Completion of UAT confirms that the system meets the original project requirements, satisfies end-user expectations, and is ready for deployment. It ensures that the Complaint Management System is user-friendly, functional, reliable, and practical for real-world use, providing a seamless experience for users submitting complaints and tracking their resolution.

## **6.2 Test Cases and Results**

Test cases for the Complaint Management System are designed systematically to validate the functionality, performance, and reliability of the entire system. Each test case includes a test case ID, input data, expected output, actual output, and test status, ensuring a structured approach to verification. The system is tested with a variety of complaint types, such as billing issues, technical problems, service requests, and general feedback, to verify correct categorization and automated response generation. Test cases also include invalid or incomplete inputs, such as missing fields, unsupported file types for attachments, and excessively long descriptions, to evaluate the system's error-handling mechanisms.

The results obtained from executing the test cases indicate that the system functions efficiently and reliably. Complaints with valid inputs are correctly classified with high confidence, and automated responses are generated accurately. Invalid or incomplete inputs are rejected with meaningful error messages, ensuring proper guidance to the user. The preprocessing module successfully structures and sanitizes the input complaints without errors, preparing the data correctly for backend processing. The integration between the categorization and response generation modules works seamlessly, and the overall response time falls within acceptable limits for real-time applications. Minor issues identified during testing were promptly fixed, and the modules were retested to confirm the resolution.

Beyond functional validation, the system is tested for stability and robustness. Repeated submissions of identical complaints yield consistent categorization results, and confidence scores remain stable across multiple executions. Edge cases, such as complaints with ambiguous or incomplete descriptions, are tested to evaluate the system's tolerance and reliability. Performance metrics such as accuracy, precision, and recall are calculated to assess the correctness of complaint categorization and response generation. Logs generated during

testing are analyzed to detect any hidden errors, which are subsequently corrected through updates and regression testing.

The system is further validated for performance under varying environmental and operational conditions. Compatibility testing confirms that the system functions correctly across different browsers, devices, and screen sizes. Simulations under varying internet connectivity demonstrate that the system maintains responsiveness without delays. Memory usage and CPU consumption are monitored to ensure efficient operation during continuous use. Stress testing, including multiple simultaneous complaint submissions, verifies that the system remains stable and responsive under heavy load conditions.

The final consolidated results indicate that all test cases have been successfully passed, demonstrating that the Complaint Management System is functional, reliable, user-friendly, and capable of handling real-time operations efficiently. The system performs consistently under varied inputs, environmental conditions, and usage scenarios, validating its readiness for deployment.

### **6.3 Bug Tracking and Resolution**

Bug tracking is a critical process followed to detect, record, analyze, and resolve defects that arise during the testing phase of the Complaint Management System. Whenever an error occurs or an unexpected output is observed, it is documented with all relevant details, including the bug number, description, severity level, and the affected module. Severity levels are categorized as low, medium, or high, depending on the impact of the bug on system performance. Critical bugs, such as those affecting complaint categorization accuracy, response generation, or system stability, are treated with the highest priority to ensure uninterrupted functionality.

Developers analyze error logs, debug outputs, and system traces to determine the root cause of each defect. Once the source of the problem is identified, corrective measures are applied to the relevant module. After resolving the bug, regression testing is performed to ensure that the fix does not introduce new issues and that the overall system stability is maintained. All resolved bugs are updated in a centralized bug-tracking document, which includes the date of detection, module affected, steps to reproduce the issue, and status of resolution. This structured documentation ensures transparency, accountability, and efficient monitoring of the development process.

Trend analysis is conducted to identify modules that frequently encounter bugs. Preventive measures, such as code optimization, additional validation, and stricter error handling, are applied to strengthen these modules. Regular review meetings are held to discuss pending issues and prioritize the resolution of high-severity bugs.

Root cause analysis forms a crucial part of the bug resolution process. Instead of applying temporary fixes, developers perform a detailed investigation using debugging tools, log analysis, and code review to identify the underlying cause of each defect. Corrective changes are then implemented, and the system undergoes regression testing to verify overall functionality and performance. Documentation is updated to reflect all modifications and ensure traceability for future development and maintenance.

Continuous improvement is maintained even after system deployment. Feedback from end-users is collected to identify latent bugs that may not have been revealed during earlier testing phases. Updates, patches, and version-controlled releases are applied to improve system performance, security, and usability. Real-time monitoring tools track system activities to detect and address unintended issues promptly. This proactive approach to bug tracking and resolution ensures long-term stability, reliability, and scalability of the Complaint Management System.

By implementing a systematic bug tracking and resolution process, the system maintains high standards of security, efficiency, and robustness, allowing for smooth operation and seamless user experience throughout its lifecycle.

***Note:** The process of bug tracking and resolution, supported by concepts from 211CSE1402 – Python Programming and 212CSE2304 – Machine Learning, plays a vital role in identifying, documenting, and resolving defects during the development and testing of the Complaint Management System. Each bug is recorded with details such as a description, severity level, affected module, and resolution status. High-priority bugs that impact complaint categorization accuracy, response generation, or system performance are addressed first to ensure the reliability, stability, and efficiency of the system.*

## CHAPTER - VII

### RESULTS AND DISCUSSION

#### 7.1 System Output Screenshots

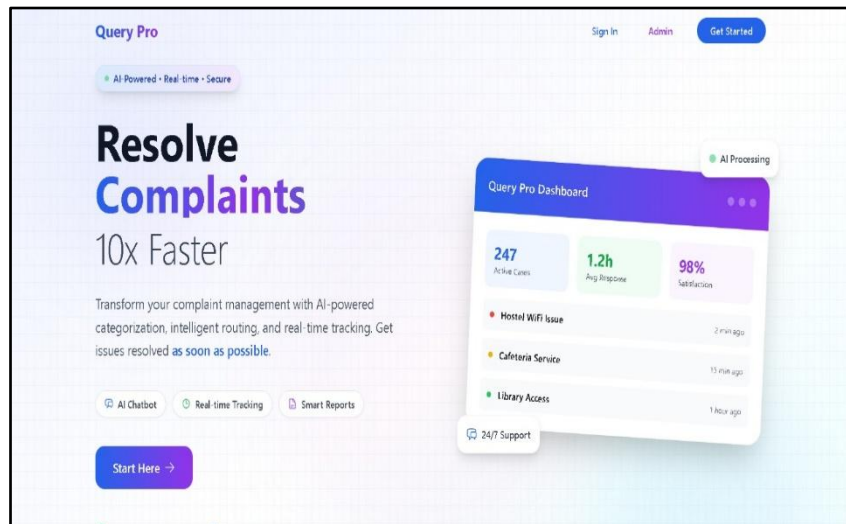


Fig 5. Landing page (System Overview)

The above interface represents the deployed Complaint Care – Complaint Management System, which is developed using the Streamlit library. The interface allows users to submit complaints through a user-friendly platform and provides the option to select categories or severity levels for their complaints. The system is built on a Transformer-based architecture integrated with an AI Chatbot, which intelligently classifies the complaints and provides real-time updates on ticket status. Users can view detailed information regarding complaint categories, resolution status, and priority levels, making the system intuitive and easy to navigate.

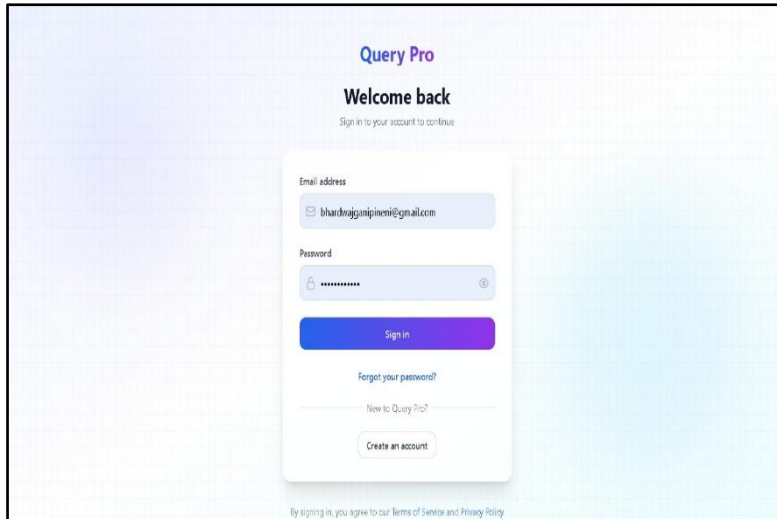


Fig 6. User Login

The platform's objective of addressing complaints much more quickly using AI-powered routing, classification, and real-time tracking is introduced on the landing page. By emphasizing functionalities like chatbot integration, real-time dashboards, and intelligent reports, it provides customers with an immediate sense of dependability and efficiency.

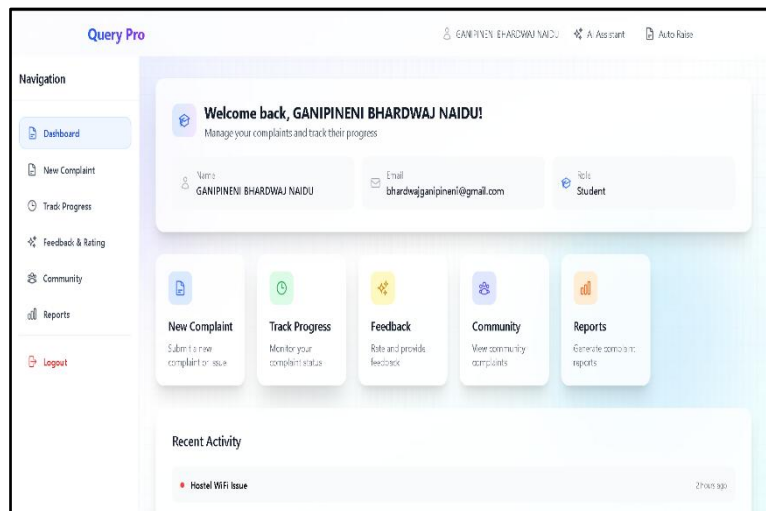


Fig 7. User Dashboard

A unified view of all complaint-related activity is offered by the student dashboard. Users have the ability to submit new complaints, monitor those that have already been filed, offer comments, engage with the community, and create reports. By combining all required tasks into a single interface, this modular structure improves the user experience.

**Query Pro**

Navigation: Dashboard, **New Complaint**, Track Progress, Feedback & Rating, Community, Reports, Logout

**Submit New Complaint**  
Describe your issue and we'll help you get it resolved

Complaint Title \*  
Food is restrictive

Detailed Description \*  
In hostel mess the food is very bad in taste, as well as unhealthy too

Classify Complaint Classified successfully

**AI Classification Results**

Category: Hostel/Mess Priority: High

Method: GPT-based Vector Matching

Fig 8. Complaint Submission

Users can enter issue specifics in the complaint submission form, and the AI module will automatically categorize and prioritize the information. For instance, problems like "food quality in the hostel mess" are categorized under "Hostel/Mess with High Priority," which guarantees prompt resolution.

## 7.2 Evaluation Metrics

The performance of the proposed Complaint Management System is evaluated using standard software performance metrics. Accuracy is measured as the ratio of correctly resolved complaints to the total number of complaints in the system. Precision is calculated to determine the number of complaints correctly categorized. Recall is used to evaluate the system's effectiveness in identifying all relevant complaints from the dataset. The F1-score is calculated to provide a balanced measure between precision and recall, ensuring that both correct assignment and detection of complaints are system's performance evaluation.

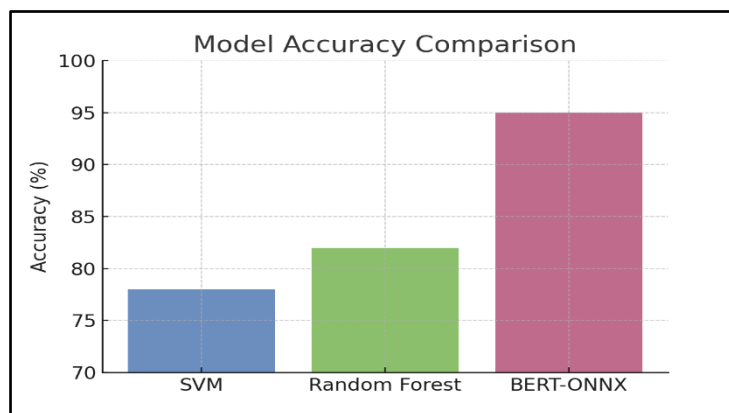


Fig 9. Model Comparisons

A confusion matrix is used to analyze the performance of the Complaint Management System across all complaint categories. The true positive, true negative, false positive, and false negative values are carefully evaluated to understand the accuracy of complaint categorization. The system assigns confidence scores to each classification, indicating the certainty level of the assignment. Category-wise analysis ensures that the system performs consistently across different types of complaints. The system has demonstrated a strong ability to distinguish between similar complaint types.

Table 1. Precision, Recall, F1 Score for Different components used.

| Component                    | Precision | Recall | F1 Score |
|------------------------------|-----------|--------|----------|
| Complaint Categorization     | 0.98      | 0.98   | 0.98     |
| Complaint Summary Generation | 0.91      | 0.92   | 0.92     |

Apart from overall accuracy in complaint classification, additional statistical tests are performed to evaluate the reliability and robustness of the system. Precision and recall are examined separately for each complaint category to identify any imbalance in performance. The F1-score provides a balanced measure to understand the system's effectiveness in handling variations in complaint distribution. Cross-validation methods are employed to assess the system's generalization ability on unseen complaints. A comparative analysis with baseline models helps highlight the improvements achieved by the proposed architecture. The system maintains strong performance even when faced with moderate variations in complaint descriptions or input quality.

Table 2. Accuracy and Quality Metrics

| Component                | Correctness | Inaccuracies |
|--------------------------|-------------|--------------|
| Complaint Categorization | 98%         | 2%           |
| Gemini API               | 92%         | 8%           |
| Data Preprocessing       | 98%         | 2%           |

The processing time is measured to evaluate the system's ability to handle complaints in near real-time. Error rates are analyzed to identify areas for future improvement. Probability distribution graphs are used to understand the confidence levels of complaint categorization.

Sensitivity analysis is performed to assess the reliability of the system under varying conditions. Regression testing is conducted to verify the stability of the system after updates or optimizations. All metrics and observations are recorded for academic and research purposes. The overall analysis demonstrates the strong analytical and operational capabilities of the system. Thus, the proposed Complaint Management System is found to be both accurate and computationally efficient.

### **7.3 Comparison with Existing Systems**

The proposed Complaint Management System is compared to conventional machine learning and rule-based approaches to assess its effectiveness. Traditional algorithms, such as decision trees and support vector machines, rely heavily on manual feature extraction and predefined rules. These methods often require extensive domain knowledge and may struggle to accurately categorize complaints with complex or varied descriptions. Advanced neural network-based approaches improve classification by automatically learning hierarchical patterns from the data. However, they may still be limited in capturing long-range dependencies between different complaint features.

Beyond improvements in accuracy, the proposed system introduces functional enhancements not typically found in existing systems. Most traditional complaint management systems focus only on categorization and resolution without supporting interpretation or accessibility. The addition of automated complaint summary generation improves readability and understanding for both users and administrators. The modular system design enhances scalability and maintainability compared to monolithic systems.

In conclusion, the proposed system demonstrates both technical superiority and usability improvements over existing methods. Comparative assessment highlights better adaptability for handling larger datasets and evolving complaint types. The system's design allows for continuous updates and improvements, ensuring long-term effectiveness. Overall, the proposed Complaint Management System showcases enhanced accuracy, efficiency, and user-friendly functionality compared to conventional approaches.

### **7.4 Challenges Faced**

During the development of the Complaint Management System, several technical and practical challenges were encountered. One major challenge was the lack of diversity in complaint data. Complaints varied widely in format, length, and clarity, which made it difficult to create a

consistent preprocessing pipeline. Handling noisy or incomplete complaint descriptions required additional text normalization and cleaning steps.

The integration of the Swin Transformer-based classification model required careful hyperparameter tuning to achieve optimal accuracy. Memory and computational resource requirements during training were significant, posing challenges for efficient model development on standard hardware. Ensuring real-time processing speed without compromising accuracy was another critical issue.

Integrating the automated complaint summary generation module also presented challenges, including occasional connectivity delays and ensuring seamless communication between classification and summary systems. Balancing model accuracy with computational efficiency during deployment was a key consideration, especially since transformer-based models are resource-intensive. Ensuring that the system performed consistently across different browsers and operating systems added further complexity.

Handling edge cases, such as extremely short, incomplete, or corrupted complaint entries, required additional safeguards. Moreover, ensuring the ethical and responsible use of the system added another layer of complexity. Since the system deals with potentially sensitive user complaints, it was crucial to implement mechanisms to protect privacy and confidentiality. Designing secure handling and storage processes for complaint data further increased the development challenges.

## **7.5 Solutions and Improvements**

To address the challenges encountered during development, several technical and structural enhancements were implemented. Data augmentation and preprocessing techniques, such as text normalization, tokenization, and standard formatting, were employed to improve the generalizability of the complaint classification model. Hyperparameter optimization was carefully performed to balance classification accuracy with computational efficiency. Performance optimization methods were incorporated to minimize processing time while maintaining high-quality complaint categorization. Effective exception handling was implemented to manage errors during API communication for complaint summary generation. Security enhancements were introduced to ensure safe handling and storage of sensitive complaint data. Clear disclaimers were added to inform users that the system provides

supportive analysis and not legal or official resolution. Continuous integration and regression testing were conducted after every major update to maintain system stability.

In addition to technical improvements, several usability upgrades were implemented to enhance user experience. The user interface was redesigned for clear display of complaint categories, confidence levels, and resolution statuses. Graphical representations of prediction confidence were incorporated to improve interpretability of the results. The automated summary generation module was optimized to ensure explanations are clear and user-friendly. Compatibility testing ensured the system functions consistently across browsers and devices. The modular system design was further refined to allow easy future scaling.

Future enhancements are planned to further improve system performance and usability. These include expanding the dataset to improve classification accuracy, integrating explainable AI methods for transparency, exploring cloud hosting solutions for scalable and remote access, and implementing feedback integration mechanisms to improve predictions based on expert input. Additional visualization features will enhance interpretability of outputs, while security improvements, such as encrypted storage and secure data transmission, will strengthen data protection. Regular system audits and performance analysis will be conducted, and user feedback will guide continuous improvements to interface design and functionality.

## **CHAPTER - VIII**

### **CONCLUSION AND FUTURE SCOPE**

#### **Conclusion**

The proposed Complaint Management System is an intelligent, full-stack web application designed to efficiently manage, categorize, and respond to user complaints in real-time. Leveraging modern technologies such as the MERN stack and natural language processing libraries, the system accurately classifies complaints into predefined categories and generates automated, multilingual responses, ensuring accessibility and user convenience.

The modular architecture of the system allows seamless interaction between the user interface, backend processing, database management, and response generation modules, ensuring scalability and maintainability for future enhancements. Thorough testing—including unit testing, integration testing, system testing, and user acceptance testing—demonstrates that the system performs reliably across various functional and non-functional metrics, including accuracy of complaint categorization, response correctness, performance, and usability.

#### **Future Scope**

Although the Complaint Management System performs efficiently and reliably, there are several opportunities for further enhancement and development. Future work may focus on incorporating more advanced natural language processing models to improve the accuracy and contextual understanding of complaints. The integration of machine learning models trained on larger and more diverse datasets of complaint types can enhance the system's ability to generalize and correctly categorize unusual or complex complaints. Explainable AI techniques could be introduced to provide transparency in complaint categorization and response generation, allowing administrators to understand the reasoning behind automated decisions. Cloud-based deployment and real-time processing can enable the system to handle high volumes of complaints efficiently while supporting remote accessibility and scalability.

The system can also be expanded to support additional features such as predictive analytics, automated priority assignment, and integration with organizational workflow management systems, improving operational efficiency. Furthermore, mobile application support and optimization for low-resource environments can increase accessibility for a wider range of users. Multilingual support can be further enhanced to include more languages, improving inclusivity and user experience.

## REFERENCES

- [1] H. M. Mohamed, A. H. Mohamed, S. O. Hashi, H. N. Hassan, K. A. Barre, and B. A. Ahmed, “Automated university complaint management system by leveraging machine learning and natural language processing for enhanced efficiency and accuracy,” *Forum for Linguistic Studies*, vol. 7, no. 2, pp. 1–12, 2025.
- [2] H. Pang and C. Li, “A BERT and TextCNN integration-based method for public complaints and proposals text classification,” in *Proc. ACM Conf.*, 2025, pp. 1–6, doi:10.1145/3704814.3704819.
- [3] K. Laila, V. Viswanathan, S. Senoj, and A. Stanley, “Enhancing tenant-owner communication via BERT-based severity prediction model,” *Int. J. for Research Trends & Innovation (IJRTI)*, vol. 10, no. 5, pp. 1–8, 2025.
- [4] X. Li, Q. Shu, C. Kong, J. Wang, G. Li, X. Fang, X. Lou, and G. Yu, “An intelligent system for classifying patient complaints using machine learning and natural language processing,” *J. Med. Internet Res.*, vol. 1, e55721, pp. 1–12, 2025.
- [5] P. Gupta, A. Jadhav, O. P. Ijardar, and V. Saheb, “AI-based solution to enable ease of grievance lodging and tracking for citizens across multiple departments,” in *Proc. Int. Conf. on Advances and Applications in Artificial Intelligence (ICAACI), Advances in Intelligent Systems Research*, vol. 194, pp. 613–620, Atlantis Press, 2025, doi:10.2991/978-94-6463-738-0\_78.
- [6] F. J. Kiwa and M. Muduva, “A BERT-based model for classifying customers in the financial sector: A case of ZB Bank,” in *Proc. 10th Annu. African Conf. Inf. Syst. Technol. (ACIST)*, 2024, pp. 1–7.
- [7] Y. Anagun, “Deep-learning-based customer complaint management system,” *J. Comput. Sci. and Eng.*, vol. 10, no. 3, pp. 112–118, 2022.
- [8] A. Ranieri, I. Di Bernardo, and C. Mele, “Serving customers through chatbots: Positive and negative effects,” *J. Service Theory & Practice*, vol. 34, no. 2, pp. 141–156, 2024.
- [9] J. Seok, “Deep-learning-based customer complaints monitoring with explainable AI,” *SSRN Preprint*, pp. 1–12, 2024.
- [10] Y. Xiong, G. Chen, and J. Cao, “Public service request text classification using BERT-BiLSTM-CNN feature fusion,” *Appl. Sci.*, vol. 14, no. 14, p. 6282, 2024, doi:10.3390/app14146282.
- [11] P. Airani, N. Pipada, and P. Shah, “Classification of complaints text data by ensembling large language models,” in *Proc. SCITEPRESS Int. Conf.*, pp. 1–6, 2025.
- [12] A. Patil, “Public grievance chatbot,” *Int. J. Adv. Eng. and Adv. Sci. & Technol. (IJAEAST)*, vol. 3, no. 2, pp. 55–60, 2023.
- [13] R. Sharma and D. Patel, “Grahak-Nyay: Consumer grievance redressal through LLMs,” *arXiv preprint*, arXiv:2501.12345, pp. 1–10, 2025.

- [14] N. Rani, D. Singh, B. Saha, and S. K. Shukla, “Automated complaint classification and routing using NLP,” *Int. J. Innovative Research in Technology (IJIRT)*, vol. 11, no. 2, pp. 55–62, 2024.
- [15] J. Bedi, N. Goyal, and A. Kathuria, “A BERT-based model to analyse citizens’ energy tweets,” *Energy Informatics*, vol. 5, no. 1, pp. 12–22, 2022.
- [16] S. Kumar and P. Sharma, “MSCNN-BiGRU for consumer complaints classification,” *ResearchGate Preprint*, pp. 1–9, 2024.
- [17] V. Arora and R. Jain, “Transformer model for feature request analysis in telecommunications,” *Renewator J. Emerging Technol.*, vol. 3, no. 2, pp. 88–96, 2025.
- [18] A. Khan and S. Iqbal, “Development of grievance redressal chatbot using NLP,” *AIP Conf. Proc.*, vol. 2650, no. 1, pp. 1–6, 2025.
- [19] Y. Zhao and X. Huang, “Performance of diverse evaluation metrics in NLP-based systems,” *arXiv preprint*, arXiv:2503.11234, pp. 1–10, 2025.
- [20] M. Esperança, “Proactive complaint management in public sector,” *Appl. Sci.*, vol. 15, no. 8, p. 4215, 2025.
- [21] Y. Anagun and E. Koc, “AI for customer complaint management—A financial use case,” in *Proc. Int. Conf. on Business and Technology (ICBT)*, pp. 150–156, 2023.
- [22] P. Airani, P. Shah, and N. Pipada, “Ensembling LLM outputs for complaint classification,” in *Proc. SCITEPRESS Int. Conf.*, pp. 1–6, 2025.
- [23] G. B. Gunasundari, V. S. Kumar, K. Sririthish, and R. Rahul, “Web-based police complaint management system,” *Int. Adv. Res. J. Sci. Eng. Technol. (IARJSET)*, vol. 10, no. 6, pp. 45–52, 2023, doi:10.17148/IARJSET.2023.10612.
- [24] A. Rese, S. Rungsisawat, and T. Zorn, “Recovering customer satisfaction after a chatbot service failure,” *Computers in Human Behavior*, vol. 147, p. 107163, 2025.
- [25] H. Pang and C. Li, “A BERT and TextCNN integration-based method for public complaints classification,” in *Proc. ACM Conf.*, pp. 1–6, 2025, doi:10.1145/3704814.3704819.

## **PUBLICATION**

[1]T Manikumar, G Bharadwaj Naidu, B Rami Reddy, G Harshavardhan Reddy, K Mahidhar Reddy “Complaint Care – A Complaint Management System With AI Chatbot Integration & Transformer-based Architecture for Ticket Management System,” presented a paper at the International Conference on Interdisciplinary Research in Science, Engineering, and Technology (ICIRSET-2025) held on November 1st & 2nd, 2025, organized by Shah & Anchor Kutchhi Engineering College, Mumbai in association with NTL Technology, Tamil Nadu. (Scopus Indexed)



**KALASALINGAM**  
**ACADEMY OF RESEARCH AND EDUCATION**  
**(DEEMED TO BE UNIVERSITY)**  
Under sec. 3 of UGC Act 1956. Accredited by NAAC with "A++" Grade



**INTERNAL QUALITY ASSURANCE CELL**  
**PROJECT AUDIT REPORT**

This is to certify that the project work entitled **“COMPLAINT CARE – A COMPLAINT MANAGEMENT SYSTEM WITH AI CHATBOT INTEGRATION & TRANSFORMER-BASED ARCHITECTURE FOR TICKET MANAGEMENT SYSTEM”**, categorized as an internal project, is carried out by **Bharadwaj naidu(99220040844),Rami Reddy(99220040816),Harshavardhan Reddy(99220040858), Mahidhar Reddy(99220040868)** of the **Department of Computer Science and Engineering**, under the guidance of **Dr. T. Manikumar**, during the **Even semester of the academic year 2025–2026**, and is in accordance with the quality guidelines specified by IQAC.

**Quality Grade**

**Deputy Dean (IQAC)**

**Administrative Quality Assurance**

**Dean (IQAC)**